

TechTribers

Java Interview Series · Part 15 · Q680–Q739

Exceptions · Inner Classes · Serialization

Throwable and finally semantics, try-with-resources, exception design and cost, inner and anonymous classes with measured GC evidence, and serialization from serialVersionUID to deserialization attacks, all run on a real JDK.

60	60	15	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `EX-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

60	7	60	15
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

⚠️	01	Exception Fundamentals	Q680–Q687	5
🔁	02	try, catch & finally in Depth	Q688–Q697	16
🔐	03	try-with-resources & AutoCloseable	Q698–Q702	27
🧭	04	Design & Best Practices	Q703–Q710	33
📚	05	Runtime Exceptions in Practice	Q711–Q714	43
🪆	06	Inner, Nested, Local & Anonymous Classes	Q715–Q727	49
📦	07	Serialization in Depth	Q728–Q739	65

© TechTribers. For personal use only. Do not redistribute.

All questions

⚠️ Exception Fundamentals

Q680 What is the Throwable hierarchy, and which exception types should you catch (or never catch)? 5

Q681 How does the compiler enforce checked exceptions? (unreported, never-thrown and reachability rules) 7

Q682 How do multi-catch and precise rethrow work, and what are their rules? 8

Q683 What is exception chaining, and how do you preserve the root cause? 10

Q684 How do you read a stack trace (frames, Caused by, ... n more, Suppressed)? 11

Q685 What does creating an exception cost, and how do you make one cheap (fillInStackTrace)? 12

Q686 Are exceptions slow? When is it acceptable to use them for control flow? 13

Q687 Why do hot NullPointerExceptions suddenly lose their stack trace? (OmitStackTraceInFastThrow) 14

🔁 try, catch & finally in Depth

Q688 What does finally really do? (return values, overriding returns, mutation puzzles) 16

Q689 What happens when an exception is thrown inside finally, and how do break, continue and return interact with it? 17

Q690 Why does catch order matter, and what is an unreachable catch clause? 18

Q691 Can you catch Errors like StackOverflowError and OutOfMemoryError, and should you? 19

Q692 How deep can recursion go before StackOverflowError, and how do you avoid it? 20

Q693 What happens when an exception is thrown in a static initialiser? (ExceptionInInitializerError and NoClassDefFoundError) 22

Q694 What happens when a constructor throws, and how do you avoid leaking resources and half-built objects? 23

Q695 How does "sneaky throw" work: why can generics throw a checked exception without declaring it? 24

Q696 How do you handle checked exceptions inside lambdas and streams? 25

Q697 How do you design a custom exception? (fields, constructors, hierarchy, cheap vs informative) 26

🔐 try-with-resources & AutoCloseable

Q698 How does try-with-resources work, and in what order are resources closed? 27

Q699 How do suppressed exceptions work, and why is try-with-resources better than try/finally? 28

Q700 How do you design an AutoCloseable class? (idempotent close, Closeable vs AutoCloseable) 30

Q701 What happens with several resources when one fails to open, and where do classic leaks still occur? 31

Q702 Which JDK types are AutoCloseable (ExecutorService, HttpClient, Files.lines...), and which aren't? 32

🧭 Design & Best Practices

Q703 Checked or unchecked: what are the guidelines, and why do modern frameworks prefer unchecked? 33

Q704 Which standard exception should you throw? (IllegalArgument, IllegalState, UnsupportedOperation, NullPointer, IndexOutOfBounds...) 35

Q705 When should you return a Result type or Optional instead of throwing? (sealed Result with pattern matching) 36

Q706 What are assertions, and how do they differ from exceptions? (assert, -ea, AssertionError) 37

Q707 What makes a good exception message, and how do getMessage, toString and getLocalizedMessage differ? 38

Q708 How do you design retries? (transient vs permanent failures, backoff, idempotency) 39

Q709 How should exceptions be translated at layer boundaries (DAO → service → API)? 40

Q710 What are the exception-handling anti-patterns, and how do you spot them? 41

📚 Runtime Exceptions in Practice

Q711 Which runtime exceptions do you see most often, and what causes each? 43

Q712 When does Java throw ArithmeticException, and how do you detect overflow? 45

Q713 What happens to exceptions in executor tasks, CompletableFuture and virtual threads? 46

Q714 How do you test exception behaviour (assertThrows) and what should the test check? 47

© TechTribers. For personal use only. Do not redistribute.

🪆 Inner, Nested, Local & Anonymous Classes

Q715 How does the compiler implement an inner class? (this$0, the hidden constructor parameter, and the JDK 18 change) 49

Q716 How do Outer.this, outer.new Inner() and name shadowing work? 50

Q717 Do inner classes, anonymous classes and lambdas cause memory leaks? (measured) 51

Q718 Static nested class vs inner class: when do you choose which, and what can each access? 53

Q719 How do local classes capture variables, and why must they be effectively final? 54

Q720 How do anonymous classes work? (constructor arguments, initializer blocks, diamond, naming) 55

Q721 Lambda vs anonymous class: how do `this`, scope, class files and identity differ? 57

Q722 How do you extend an inner class from outside, and what is outer.super()? 58

Q723 Can inner classes have static members? What changed in Java 16 (and local records, enums, interfaces)? 59

Q724 What are the implicit modifiers of nested types in interfaces and enums? 60

Q725 What is double-brace initialization, and why is it considered harmful? 61

Q726 How do generic inner classes work (Outer<T>.Inner) and should a collection's Node be inner or static? 62

Q727 What are the names and flags of nested, local, anonymous and lambda classes (getName, getSimpleName, isMemberClass...)? 63

📦 Serialization in Depth

Q728 What exactly is serialized, and which constructors run on deserialization? 65

Q729 What does serialVersionUID do in practice? (a real InvalidClassException) 66

Q730 How do you evolve a serializable class? (compatible vs incompatible changes, missing fields) 68

Q731 How does custom serialization work (writeObject, readObject, defaultWriteObject, validation)? 69

Q732 What are readResolve, writeReplace and the serialization proxy pattern? 70

Q733 Externalizable vs Serializable: when would you use it? 72

Q734 How does serialization handle shared references, cycles and repeated writes (reset, writeUnshared, deep copy)? 73

Q735 How big and how fast is Java serialization, and what are the alternatives? 74

Q736 Why is deserialization dangerous, and how do you harden it? (code runs before the cast) 76

Q737 How are records, enums and lambdas serialized? 77

Q738 Why is serializing inner, anonymous and local classes a bad idea? 78

Q739 How do you debug NotSerializableException, and which JDK types are not serializable? 79

© TechTribers. For personal use only. Do not redistribute.

Section 01

Exception Fundamentals

The Throwable hierarchy, how the compiler enforces checked exceptions, multi-catch and precise rethrow, chaining, stack traces, and what exceptions really cost.

Questions 680–687 · 8 questions



🌳 Q680 EX-001 • Basic

What is the Throwable hierarchy, and which exception types should you catch (or never catch)?

⚡ Quick answer

`Throwable` has two branches: `Error` (serious JVM or environment problems: `OutOfMemoryError`, `StackOverflowError`, `LinkageError`, `AssertionError`) and `Exception`. Under `Exception`, `RuntimeException` and its subclasses are unchecked (programming errors such as `NullPointerException`, `IllegalArgumentException`); every other `Exception` is **checked** and must be caught or declared. `Throwable` itself is also checked. Catch the **most specific** exception you can actually handle; avoid catching `Throwable`, `Error` and (usually) bare `Exception`.

The rule the compiler applies: a class is **checked** if it is `Throwable` or `Exception` or a subclass of either **that is not** under `RuntimeException` or `Error`. Note that `Exception` and `Throwable` themselves are checked, which is why `throw new Exception()` needs a `throws` clause.

What each branch means in practice:

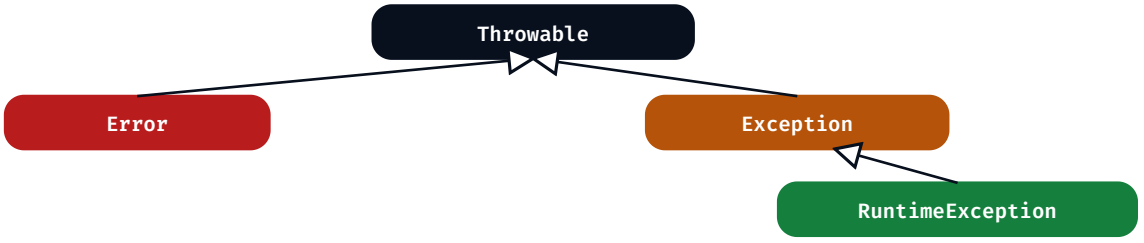
- `Error`: the JVM or the environment is in trouble. Usually you can't recover, and code in a `catch (Error)` can't be trusted to run (an `OutOfMemoryError` handler that allocates fails again). Let them propagate, log, and let the process or thread die.
- `RuntimeException`: a bug in the caller or callee (null, bad argument, wrong state). Fix the code; don't catch these to continue, except at a boundary (a request handler, a worker loop).
- **Checked** `Exception`: an expected, possibly recoverable failure outside the program's control (I/O, network, parsing external data).

© TechTribers. For personal use only. Do not redistribute.

```java
Class<?>[] types = {StackOverflowError.class, OutOfMemoryError.class, AssertionError.class,
NullPointerException.class, IllegalArgumentException.class, IOException.class,
        java.sql.SQLException.class, InterruptedException.class, TimeoutException.class,
ExecutionException.class, Exception.class, Error.class, Throwable.class};
for (Class<?> c : types) {
    String kind = Error.class.isAssignableFrom(c) ? "Error (unchecked)" :
RuntimeException.class.isAssignableFrom(c) ? "RuntimeException (unchecked)" : "checked";
    System.out.println(c.getSimpleName() + " -> " + kind + ", superclass " +
c.getSuperclass().getSimpleName());
}
```

▶ Output

```
StackOverflowError -> Error (unchecked), superclass VirtualMachineError
OutOfMemoryError -> Error (unchecked), superclass VirtualMachineError
AssertionError -> Error (unchecked), superclass Error
NullPointerException -> RuntimeException (unchecked), superclass RuntimeException
IllegalArgumentException -> RuntimeException (unchecked), superclass RuntimeException
IOException -> checked, superclass Exception
SQLException -> checked, superclass Exception
InterruptedException -> checked, superclass Exception
TimeoutException -> checked, superclass Exception
ExecutionException -> checked, superclass Exception
Exception -> checked, superclass Throwable
Error -> Error (unchecked), superclass Throwable
Throwable -> checked, superclass Object
```



Error: unchecked
StackOverflowError
OutOfMemoryError
AssertionError
LinkageError

Exception (not Runtime): checked
IOException
SQLException
InterruptedException
TimeoutException

RuntimeException: unchecked
NullPointerException
IllegalArgument / IllegalState
ClassCastException
IndexOutOfBounds · Arithmetic

Catch Exception types you can actually handle. Catching Throwable or Error is almost always wrong.

💡 **Pro tip**

A strong one-liner: "Error: don't catch. RuntimeException: fix the bug. Checked: handle it or declare it."

© TechTribers. For personal use only. Do not redistribute.

Q681 EX-002 • Intermediate

How does the compiler enforce checked exceptions? (unreported, never-thrown and reachability rules)

⚡ **Quick answer**

The compiler checks three things: (1) a statement that can throw a **checked** exception must be inside a `try` that catches it (or a supertype) or in a method that **declares** it with `throws`; (2) a `catch` of a **checked** exception type that the `try` body **cannot throw** is an error ("exception X is never thrown"), but `catch (Exception)` and unchecked types are always allowed; (3) `throw` of a checked type counts as a thrown exception. Unchecked exceptions are never checked.

Each line above was compiled for real by the JDK's in-memory compiler (`javax.tools`), so the messages are exactly what `javac` prints.

Reading them: `new FileReader("x")` declares `throws FileNotFoundException`, so you must handle it; `catch (IOException)` around code that can't throw it is rejected (protecting you from dead handlers); `catch (Exception)` and `catch (RuntimeException)` always compile because they may legitimately catch unchecked exceptions. Checked exceptions are a **compile-time** feature only: the JVM itself doesn't distinguish them (see the sneaky-throw question).

Java

```
static String compile(String body) {                    // compile a snippet in memory and report
the compiler's first error line
    javax.tools.JavaCompiler jc = javax.tools.ToolProvider.getSystemJavaCompiler();
    javax.tools.DiagnosticCollector<javax.tools.JavaFileObject> dc = new
javax.tools.DiagnosticCollector<>();
    javax.tools.JavaFileObject src = new
javax.tools.SimpleJavaFileObject(java.net.URI.create("string:///T.java"),
javax.tools.JavaFileObject.Kind.SOURCE) {
        @Override public CharSequence getCharContent(boolean ignore) { return "import java.io.*;
public class T { " + body + " }"; }
    };
    boolean ok = jc.getTask(null, jc.getStandardFileManager(dc, null, null), dc, List.of("-
proc:none", "-d", System.getProperty("java.io.tmpdir")), null, List.of(src)).call();
    StringBuilder sb = new StringBuilder(ok ? "compiles" : "");
    for (javax.tools.Diagnostic<? extends javax.tools.JavaFileObject> d : dc.getDiagnostics())
        if (d.getKind() == javax.tools.Diagnostic.Kind.ERROR)
sb.append(d.getMessage(Locale.ENGLISH).split("\n")[0]);
    return sb.toString();
}
String[] cases = {
    "void f() { new FileReader(\"x\"); }",
    "void f() throws IOException { new FileReader(\"x\"); }",
    "void f() { try { int i = 1; } catch (IOException e) { } }",
    "void f() { try { } catch (RuntimeException e) { } }",
    "void f() { try { } catch (Exception e) { } }",
    "void f() { throw new RuntimeException(); }",
    "void f() { throw new Exception(); }",
    "int f() { try { return 1; } finally { } }"
```


© TechTribers. For personal use only. Do not redistribute.

```
};
for (String c : cases) System.out.println(c + "\n   -> " + compile(c));
```

▶ Output

```
void f() { new FileReader("x"); }
   -> unreported exception java.io.FileNotFoundException; must be caught or declared to be thrown
void f() throws IOException { new FileReader("x"); }
   -> compiles
void f() { try { int i = 1; } catch (IOException e) { } }
   -> exception java.io.IOException is never thrown in body of corresponding try statement
void f() { try { } catch (RuntimeException e) { } }
   -> compiles
void f() { try { } catch (Exception e) { } }
   -> compiles
void f() { throw new RuntimeException(); }
   -> compiles
void f() { throw new Exception(); }
   -> unreported exception java.lang.Exception; must be caught or declared to be thrown
int f() { try { return 1; } finally { } }
   -> compiles
```

💡 **Pro tip**

When an interviewer asks "why does this not compile?", name the rule: unreported checked exception, or a catch for a checked type that is never thrown.

Q682 EX-003 • Intermediate

How do multi-catch and precise rethrow work, and what are their rules?

⚡ **Quick answer**

Multi-catch (`catch (A | B e)`) handles unrelated exception types with one block; the alternatives **may not be related by subclassing** and the parameter `e` is implicitly `final`. **Precise rethrow** (Java 7): if you catch `Exception` (or `Throwable`) and rethrow the **unmodified, effectively final** parameter, the compiler knows the real set of checked types the `try` body can throw, so your method needs to declare only those.

Reading the cases: case 1 is rejected (related alternatives), case 2 is rejected (the multi-catch parameter is implicitly final), case 3 compiles, and case 4 doesn't. Case 3 compiles because `e` is only rethrown (the compiler tracks that the body throws just `FileNotFoundException`); case 4 copies `e` into another variable first, which turns the thrown type back into `Exception`, so the `throws FileNotFoundException` declaration is no longer enough. Use multi-catch when the handling is identical; if you need different handling, use separate blocks.

Java

```
static String compile(String body) {                    // compile a snippet in memory and report
the compiler's first error line
    javax.tools.JavaCompiler jc = javax.tools.ToolProvider.getSystemJavaCompiler();
    javax.tools.DiagnosticCollector<javax.tools.JavaFileObject> dc = new
```


© TechTribers. For personal use only. Do not redistribute.

```
javax.tools.DiagnosticCollector<>();
    javax.tools.JavaFileObject src = new
javax.tools.SimpleJavaFileObject(java.net.URI.create("string:///T.java"),
javax.tools.JavaFileObject.Kind.SOURCE) {
        @Override public CharSequence getCharContent(boolean ignore) { return "import java.io.*;
public class T { " + body + " }"; }
    };
    boolean ok = jc.getTask(null, jc.getStandardFileManager(dc, null, null), dc, List.of("-
proc:none", "-d", System.getProperty("java.io.tmpdir")), null, List.of(src)).call();
    StringBuilder sb = new StringBuilder(ok ? "compiles" : "");
    for (javax.tools.Diagnostic<? extends javax.tools.JavaFileObject> d : dc.getDiagnostics())
        if (d.getKind() == javax.tools.Diagnostic.Kind.ERROR)
sb.append(d.getMessage(Locale.ENGLISH).split("\n")[0]);
    return sb.toString();
}
String[] cases = {
    "void f() { try { throw new FileNotFoundException(); } catch (FileNotFoundException |
IOException e) { } }",
    "void f() { try { throw new FileNotFoundException(); } catch (FileNotFoundException |
IllegalStateException e) { e = null; } }",
    "void f() throws FileNotFoundException { try { throw new FileNotFoundException(); } catch
(Exception e) { throw e; } }",
    "void f() throws FileNotFoundException { try { throw new FileNotFoundException(); } catch
(Exception e) { Exception x = e; throw x; } }"
};
for (int i = 0; i < cases.length; i++) System.out.println("case " + (i + 1) + ": " +
compile(cases[i]));
static void parse(String s) {
    try { Integer.parseInt(s.trim()); }
    catch (NumberFormatException | NullPointerException e) {
System.out.println(e.getClass().getSimpleName() + " handled by one block"); }
}
parse("x"); parse(null); parse(" 5 ");
```

▶ **Output**

```
case 1: Alternatives in a multi-catch statement cannot be related by subclassing
case 2: multi-catch parameter e may not be assigned
case 3: compiles
case 4: unreported exception java.lang.Exception; must be caught or declared to be thrown
NumberFormatException handled by one block
NullPointerException handled by one block
```

⚠️ **Interview trap**

Multi-catch doesn't give you a union type with the members of both: inside the block `e` has the **common supertype**, so you can only call methods that exist on that supertype.

© TechTribers. For personal use only. Do not redistribute.

Q683 EX-004 • Intermediate

What is exception chaining, and how do you preserve the root cause?

⚡ Quick answer

Wrap a low-level exception in a higher-level one and pass the original as the **cause**: `new IllegalStateException("cannot save report", e)`, or `initCause(e)` once. The cause appears as `Caused by:` in the stack trace and is available through `getCause()`. Dropping the cause (`throw new X(e.getMessage())`) discards the stack trace of the real failure, the most common mistake in exception translation.

Guidelines: always pass the cause; add **new information** in the wrapper's message (what the code was trying to do), not a copy of the cause's message; wrap at **layer boundaries** (data access → service → API), not at every method. A root-cause helper (loop on `getCause()`) is common in logging and error mapping; guard against cycles in untrusted chains. `initCause` can be called only once and only if the cause wasn't set in the constructor.

Java

```
static void low() throws IOException { throw new IOException("disk full"); }
static void mid() { try { low(); } catch (IOException e) { throw new IllegalStateException("cannot
save report", e); } }
static Throwable rootCause(Throwable t) { while (t.getCause() != null) t = t.getCause(); return t; }
try { mid(); } catch (IllegalStateException e) {
    System.out.println(e.getMessage() + " | cause: " + e.getCause() + " | root: " +
rootCause(e).getClass().getSimpleName());
    StringWriter sw = new StringWriter();
    e.printStackTrace(new PrintWriter(sw));
    sw.toString().lines().map(l -> l.replaceAll("\\(.*\\)", "")).filter(l -> l.startsWith("java.")
|| l.startsWith("Caused by") || l.contains("at Main.")).forEach(System.out::println);
}
Throwable t = new RuntimeException("outer");
t.initCause(new Error("late cause"));
try { t.initCause(new Error("again")); } catch (IllegalStateException e) {
System.out.println("initCause only once: " + e.getMessage().replace("java.lang.RuntimeException: ",
""")); }
RuntimeException lost = new RuntimeException("wrapped by message only: " + new
IOException("original").getMessage());
System.out.println("cause preserved? " + (lost.getCause() != null));
```

▶ Output

```
cannot save report | cause: java.io.IOException: disk full | root: IOException
java.lang.IllegalStateException: cannot save report
        at Main.mid
        at Main.main
Caused by: java.io.IOException: disk full
        at Main.low
initCause only once: Can't overwrite cause with java.lang.Error: again
cause preserved? false
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`new RuntimeException(e)` (cause only) sets the message to `e.toString()`. Fine for propagation, but a hand-written message is better for diagnosing which operation failed.

📜 **Q684** EX-005 • Basic

How do you read a stack trace (frames, Caused by, ... n more, Suppressed)?

⚡ **Quick answer**

A trace lists the exception class and message, then the **frames** from where it was created (top) outward to the thread's entry point. Each frame is `package.Class.method(File.java:line)`. `Caused by:` introduces the wrapped exception with its own frames; `... 12 more` means those frames are identical to the enclosing trace; `Suppressed:` lists exceptions attached by try-with-resources. Read the **first frame in your own code** at the top of each block, and the deepest `Caused by` for the root cause.

How to use it: (1) read the **top message** (what), (2) find the **first frame in your package** (where), (3) read the **last** `Caused by` (why), (4) check `Suppressed` for secondary failures. Frames from `jdk.internal.*`, frameworks and proxies are noise; your own frames and the exception's creation point are the signal.

Programmatic access: `getStackTrace()` returns `StackTraceElement[]` (class, method, file, line); `StackWalker` (Java 9) is the efficient way to inspect the current stack without creating an exception. Line numbers need debug info (`-g`, the default for `javac`).

Java

```
static String clean(Throwable t) {                                    // print a trace
without JDK launcher frames or varying line numbers
    StringWriter sw = new StringWriter(); t.printStackTrace(new PrintWriter(sw));
    return sw.toString().lines().filter(l -> !l.trim().startsWith("at java.base") &&
!l.trim().startsWith("at jdk.") && !l.trim().startsWith("..."))
            .map(l -> l.replaceAll("\\(Main.java:\\d+\\)", "
(Main.java:N)")).collect(Collectors.joining("\n"));
}
static void level3() { throw new IllegalArgumentException("bad id"); }
static void level2() { try { level3(); } catch (RuntimeException e) { throw new
IllegalStateException("load failed", e); } }
static void level1() {
    try { level2(); }
    catch (RuntimeException e) { RuntimeException top = new RuntimeException("request failed", e);
top.addSuppressed(new IllegalStateException("cleanup also failed")); throw top; }
}
try { level1(); }
catch (RuntimeException e) {
    System.out.println(clean(e));
    StackTraceElement first = e.getStackTrace()[0];
    System.out.println(first.getClassName() + "." + first.getMethodName() + " line known: " +
(first.getLineNumber() > 0) + ", frames: " + (e.getStackTrace().length >= 2));
}
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
java.lang.RuntimeException: request failed
        at Main.level1(Main.java:N)
        at Main.main(Main.java:N)
        Suppressed: java.lang.IllegalStateException: cleanup also failed
Caused by: java.lang.IllegalStateException: load failed
        at Main.level2(Main.java:N)
        at Main.level1(Main.java:N)
Caused by: java.lang.IllegalArgumentException: bad id
        at Main.level3(Main.java:N)
Main.level1 line known: true, frames: true
```

💡 **Pro tip**

A trace with **no frames at all** is the JVM's fast-throw optimisation (see the dedicated question): search the logs for the first occurrence.

Q685 EX-006 • Advanced

What does creating an exception cost, and how do you make one cheap (fillInStackTrace)?

⚡ **Quick answer**

Most of an exception's cost is `fillInStackTrace()`, which walks the stack and records every frame when the object is **constructed** (not when it is thrown). Measured on JDK 25: about **640 ns** per throw+catch with a trace one frame deep, **1,500 ns** with ten more frames and **8,600 ns** with a hundred, versus **17–500 ns** when the trace is disabled. For expected failures on a hot path, use an exception class created with `writableStackTrace = false` (or override `fillInStackTrace`).

Measured here (ns per throw+catch, round 3 of 3, JDK 25; "stack" = extra frames above the throw):

Stack	With trace	Without trace
1	638	17
10	1,517	461
100	8,624	2,541

The trace-less cost isn't zero: the JVM still **unwinds** the stack, and that grows with depth. Use cheap exceptions only for **expected, frequent, non-diagnostic** conditions (parsing user input in a tight loop, control flow in a parser). For genuine errors keep the trace: it is the data you will need at 3 a.m.

Java

```
static class Lightweight extends RuntimeException { Lightweight(String m) { super(m, null, false,
false); } }     // message, cause, enableSuppression, writableStackTrace
static class Overriding extends RuntimeException {
    Overriding(String m) { super(m); }
```

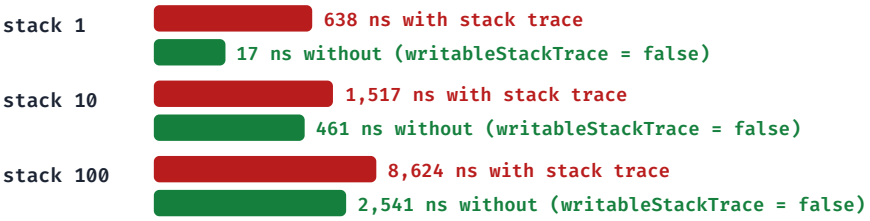

© TechTribers. For personal use only. Do not redistribute.

```
    @Override public synchronized Throwable fillInStackTrace() { return this; }
// the classic trick: skip the stack walk
}
static void deep(int n) { if (n == 0) throw new RuntimeException("full"); deep(n - 1); }
try { deep(50); } catch (RuntimeException e) { System.out.println("normal exception has the frames:
" + (e.getStackTrace().length >= 50)); }
Lightweight l = new Lightweight("no trace"); Overriding o = new Overriding("overridden");
System.out.println(l.getStackTrace().length + " " + o.getStackTrace().length + " " + l.getMessage()
+ " " + o.getMessage());
l.addSuppressed(new Exception("ignored"));
System.out.println(l.getSuppressed().length + " (suppression disabled too)");
```

▶ Output

```
normal exception has the frames: true
0 0 no trace overridden
0 (suppression disabled too)
```

Cost of one throw + catch (JDK 25, ns, round 3 of 3; stack = extra frames above the throw)



Filling in the stack trace dominates: it walks and records every frame. Bars are logarithmic.

5,000,000 parseInt calls, half invalid: 1,156 ms via NumberFormatException vs 31 ms with validation first.

⚠️ **Interview trap**

A trace-less exception makes debugging impossible if it ever escapes to a log. Use it only where the catching code always handles it locally, and name the class clearly (`ValidationFailure` , not `Oops`).

⏱️ **Q686** EX-007 • Intermediate

Are exceptions slow? When is it acceptable to use them for control flow?

⚡ **Quick answer**

A throw+catch costs microseconds (dominated by the stack trace), roughly **30–40× slower** than a validation branch for rejecting bad input: measured 5 million `parseInt` calls with half invalid took **1,156 ms** using exceptions and **31 ms** with a pre-check. When failures are **rare**, exceptions are free in the happy path (the JIT compiles `try` blocks to zero-cost tables). Use return values, `Optional` or validation for **expected, frequent** outcomes, and exceptions for **exceptional** ones.

Measured (JDK 25): 5,000,000 parses, half invalid: **1,156 ms** with exceptions vs **31 ms** with validation, a factor of about 37. In an all-valid workload the two are equal. The cost is proportional to the **failure rate**.

© TechTribers. For personal use only. Do not redistribute.

When exceptions are fine: file not found, network down, a malformed request at the API boundary, unexpected states (rare by definition). When they hurt: a parser that uses exceptions to try alternatives per token, `Integer.parseInt` to probe whether a string is numeric in a loop, `Optional.get` on `NoSuchElement` as control flow.

The deeper argument against control-flow exceptions isn't speed but **clarity**: a reader expects `catch` to mean something went wrong.

Java

```
static boolean isInt(String s) { if (s.isEmpty()) return false; for (int i = 0; i < s.length(); i++)
if (!Character.isDigit(s.charAt(i))) return false; return true; }
int bad1 = 0, bad2 = 0;
for (int i = 0; i < 1000; i++) {
    String s = i % 2 == 0 ? "12" : "x";
    try { Integer.parseInt(s); } catch (NumberFormatException e) { bad1++; }          // exception
path
    if (!isInt(s)) bad2++;                                                            // validation
path
}
System.out.println(bad1 + " " + bad2 + " " + (bad1 == bad2));
```

▶ Output

```
500 500 true
```

💡 **Pro tip**

Measure with JMH, not a loop in `main` : the JIT can inline or eliminate pieces of micro-benchmarks. The ratio above is the order of magnitude to remember.

Q687 EX-008 • Advanced

Why do hot NullPointerExceptions suddenly lose their stack trace? (OmitStackTraceInFastThrow)

⚡ **Quick answer**

When the JIT (C2) compiles code that throws **implicit** exceptions repeatedly (`NullPointerException` , `ArrayIndexOutOfBoundsException` , `ArithmeticException` , `ClassCastException` , `ArrayStoreException`), it replaces the throw with a **preallocated exception that has an empty stack trace**. This is the default (`-XX:+OmitStackTraceInFastThrow`). In a measured run, the first stack-trace-less NPE appeared at iteration **5,121**, and 206,815 of 300,000 NPEs had no trace; with `-XX:-OmitStackTraceInFastThrow` all 300,000 had full traces.

Measured on JDK 25 with 300,000 iterations: 93,185 traced, 206,815 trace-less, first trace-less at iteration 5,121. The exact threshold varies with JIT timing, but the behaviour is reliable once the method is hot.

Symptom: logs show `java.lang.NullPointerException` with no `at ...` lines after the application has run for a while. The *first* occurrences (before compilation) have full traces, so search the logs from the start of the incident. **Fixes:** find the first traced occurrence; for debugging add `-XX:-OmitStackTraceInFastThrow` ; or fix the

© TechTribers. For personal use only. Do not redistribute.

bug, which makes the question moot. The explicit `throw new NullPointerException()` in your own code is a normal object and is not affected: only exceptions thrown implicitly by the JVM.

Java

```
static int len(String s) { return s.length(); }
int withTrace = 0, withoutTrace = 0, firstEmpty = -1;
for (int i = 0; i < 2_000_000; i++) {
    try { len(null); }
    catch (NullPointerException e) {
        if (e.getStackTrace().length == 0) { withoutTrace++; if (firstEmpty < 0) firstEmpty = i; }
else withTrace++;
    }
}
System.out.println("some NPEs had full traces: " + (withTrace > 0) + ", some had none: " +
(withoutTrace > 0));
```

▶ Output

```
some NPEs had full traces: true, some had none: true
```

Hot NPEs lose their stack trace (C2 OmitStackTraceInFastThrow, on by default)



Measured: of 300,000 identical NPEs, 93,185 had a trace and 206,815 had none.

With -XX:-OmitStackTraceInFastThrow: all 300,000 had full traces.

Symptom in production: an old log line shows the exception with NO stack trace. Look earlier in the logs for the first one.

⚠ **Interview trap**

Don't leave `-XX:-OmitStackTraceInFastThrow` on permanently in a system that throws a lot: the optimisation exists to avoid the cost of filling traces in tight failure loops.

© TechTribers. For personal use only. Do not redistribute.

Section 02

try, catch & finally in Depth

The puzzles and semantics: what finally really does, masking, catch order, catching Errors, static initialisers, constructors, sneaky throw, lambdas, and custom exceptions.

Questions 688–697 · 10 questions



Q688 EX-009

• Intermediate

What does finally really do? (return values, overriding returns, mutation puzzles)

⚡ Quick answer

`finally` runs after the `try` / `catch` completes, **including after a** `return` . The `return` expression is **evaluated and saved before** `finally` runs, so changing a primitive in `finally` doesn't change the returned value; but if `finally` itself does `return` (or throws), it **replaces** the result and silently discards any pending exception.

Line by line: `f2` returns **1** (the value of `x` was copied before `finally` set it to 2); `f3` returns **2** (finally's `return` wins); `f4` returns **4** and the `RuntimeException` vanishes without a trace; `f5` returns `[1, 2]` because the **same list object** was returned and mutated; `f6` returns `ab` because the `String` was already created; `f7` returns **2**.

Rule: **never** `return` **or** `throw` **from** `finally` . It breaks the contract of the surrounding code, hides exceptions and is a common static-analysis finding (`finally block does not complete normally`).

Java

```
static int f1() { try { return 1; } finally { System.out.print("[finally] "); } }
static int f2() { int x = 1; try { return x; } finally { x = 2; } }
static int f3() { try { return 1; } finally { return 2; } }
static int f4() { try { throw new RuntimeException("lost"); } finally { return 4; } }
static List<Integer> f5() { List<Integer> l = new ArrayList<>(List.of(1)); try { return l; } finally
{ l.add(2); } }
static String f6() { StringBuilder sb = new StringBuilder("a"); try { return
sb.append("b").toString(); } finally { sb.append("c"); } }
static int f7() { int i = 0; try { i = 1; throw new Exception(); } catch (Exception e) { i = 2;
return i; } finally { i = 3; } }
System.out.println(f1());
System.out.println(f2() + " " + f3() + " " + f4() + " " + f5() + " " + f6() + " " + f7());
```

▶ Output

```
[finally] 1
1 2 4 [1, 2] ab 2
```


© TechTribers. For personal use only. Do not redistribute.




```
x = 1; try { return x; } finally { x = 2; } → returns 1 (the primitive was copied before finally)
try { return 1; } finally { return 2; } → returns 2 (a return in finally REPLACES the result)
try { throw e; } finally { return 4; } → returns 4 (and the exception is silently LOST)
List l; try { return l; } finally { l.add(2); } → the caller sees the mutation (same object)
```

⚠️ **Interview trap**

The classic interview puzzle has `int x = 1; try { return x; } finally { x++; }`. The answer is 1, because primitives are copied at the `return`.

Q689 EX-010 • Intermediate

What happens when an exception is thrown inside finally, and how do break, continue and return interact with it?

⚡ **Quick answer**

An exception thrown in `finally` **replaces** the one in flight (the original is lost unless you attach it). `break`, `continue` and `return` inside the `try` also trigger `finally`, which runs once per exit. To avoid masking, keep `finally` blocks small, **catch cleanup failures inside** `finally`, or use try-with-resources, which records the second failure as **suppressed**.

`masks()` shows the loss: the caller sees only "from finally" and no trace of "original". In the loop, `finally` ran for `i = 0` (normal), `i = 1` (`continue`) and `i = 2` (`break`), so `count` is 13. `safe()` is the defensive pattern: any cleanup that can fail is wrapped so the original exception survives.

When `finally` does **not** run: `System.exit()` or `Runtime.halt()` before it, a JVM crash or kill signal, an infinite loop or deadlock in the `try`, or the thread being killed. A daemon thread is simply abandoned when the JVM exits.

Java

```
static void masks() {
    try { try { throw new IllegalStateException("original"); } finally { throw new
RuntimeException("from finally"); } }
    catch (RuntimeException e) { System.out.println(e.getMessage() + " | suppressed=" +
e.getSuppressed().length + " | cause=" + e.getCause()); }
}
masks();
int count = 0;
for (int i = 0; i < 3; i++) {
    try { if (i == 1) continue; if (i == 2) break; count += 10; }
    finally { count++; System.out.print("finally#" + i + " "); }
}
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(count);
static int safe() {
    try { throw new IllegalStateException("original"); }
    finally { try { Integer.parseInt("x"); } catch (NumberFormatException ignored) {
System.out.println("cleanup error handled inside finally"); } }
}
try { safe(); } catch (IllegalStateException e) { System.out.println("still " + e.getMessage()); }
```

▶ Output

```
from finally | suppressed=0 | cause=null
finally#0 finally#1 finally#2 13
cleanup error handled inside finally
still original
```

💡 Pro tip

If a cleanup call can throw, prefer try-with-resources: the original stays primary and the cleanup failure is added with `addSuppressed`.

Q690 EX-011 • Basic

Why does catch order matter, and what is an unreachable catch clause?

⚡ Quick answer

`catch` clauses are tested **top to bottom** and the first matching type wins, so a **more general** type must come **after** a more specific one. Putting a superclass first makes the later clause unreachable, which is a **compile error** ("exception X has already been caught"). Order from most specific to most general, and don't catch a general type unless you handle it differently.

The compiler messages come from the real compiler (cases 1 and 2 are errors, 3 and 4 compile). Order matters for behaviour even when it compiles: with unrelated or sibling types the order doesn't change the result, but with a subtype relationship it does. A common design is: specific handlers that recover, then one general handler that logs and wraps.

Java

```
static String compile(String body) {                    // compile a snippet in memory and report
the compiler's first error line
    javax.tools.JavaCompiler jc = javax.tools.ToolProvider.getSystemJavaCompiler();
    javax.tools.DiagnosticCollector<javax.tools.JavaFileObject> dc = new
javax.tools.DiagnosticCollector<>();
    javax.tools.JavaFileObject src = new
javax.tools.SimpleJavaFileObject(java.net.URI.create("string:///T.java"),
javax.tools.JavaFileObject.Kind.SOURCE) {
        @Override public CharSequence getCharContent(boolean ignore) { return "import java.io.*;
public class T { " + body + " }"; }
    };
    boolean ok = jc.getTask(null, jc.getStandardFileManager(dc, null, null), dc, List.of("-
proc:none", "-d", System.getProperty("java.io.tmpdir")), null, List.of(src)).call();
    StringBuilder sb = new StringBuilder(ok ? "compiles" : "");
```


© TechTribers. For personal use only. Do not redistribute.

```
    for (javax.tools.Diagnostic<? extends javax.tools.JavaFileObject> d : dc.getDiagnostics())
        if (d.getKind() == javax.tools.Diagnostic.Kind.ERROR)
sb.append(d.getMessage(Locale.ENGLISH).split("\n")[0]);
    return sb.toString();
}
String[] cases = {
    "void f() { try { throw new FileNotFoundException(); } catch (IOException e) { } catch
(FileNotFoundException e) { } }",
    "void f() { try { int x = 1 / 0; } catch (Exception e) { } catch (RuntimeException e) { } }",
    "void f() { try { throw new FileNotFoundException(); } catch (FileNotFoundException e) { } catch
(IOException e) { } }",
    "void f() { try { int x = 1 / 0; } catch (ArithmeticException e) { } catch (RuntimeException e)
{ } catch (Exception e) { } }"
};
for (int i = 0; i < cases.length; i++) System.out.println("case " + (i + 1) + ": " +
compile(cases[i]));
try { throw new FileNotFoundException("missing.txt"); }
catch (FileNotFoundException e) { System.out.println("most specific handler: " + e.getMessage()); }
catch (IOException e) { System.out.println("general I/O handler"); }
```

▶ Output

```
case 1: exception java.io.FileNotFoundException has already been caught
case 2: exception java.lang.RuntimeException has already been caught
case 3: compiles
case 4: compiles
most specific handler: missing.txt
```

⚠️ **Interview trap**

`catch (Exception e)` followed by a `catch` of any `Exception` subtype is always an error, while `catch (Throwable)` first would swallow everything including `Error` s.

🔥 **Q691** EX-012 • Advanced

Can you catch Errors like StackOverflowError and OutOfMemoryError, and should you?

⚡ **Quick answer**

You *can* catch any `Throwable` , and sometimes it is safe: a `StackOverflowError` unwinds the stack by itself, so after the catch the thread continues with plenty of stack. An `OutOfMemoryError` from one **oversized allocation** (`new long[Integer.MAX_VALUE]`) leaves the heap fine. But after a heap-exhaustion OOM or a `LinkageError` , program state may be corrupt, so the standard advice is: log and terminate (or fail the request), don't try to continue.

Where catching `Throwable` is legitimate: the **top level** of a thread, request loop or executor task, to log and keep the process alive (the framework does exactly this), and in shutdown hooks. Where it isn't: business code that "handles" `OutOfMemoryError` by trying again.

Practical advice for OOM in production: `-XX:+HeapDumpOnOutOfMemoryError` , `-XX:+ExitOnOutOfMemoryError` (let the orchestrator restart the pod) and alert on it, instead of catching and limping on. See the JVM part for the

© TechTribers. For personal use only. Do not redistribute.

different kinds of OOM.

```java
static int depth;
static void recurse() { depth++; recurse(); }
try { recurse(); } catch (StackOverflowError e) { System.out.println("caught StackOverflowError,
thread continues, depth > 1000: " + (depth > 1000)); }
try { long[] huge = new long[Integer.MAX_VALUE]; System.out.println(huge.length); }
catch (OutOfMemoryError e) { System.out.println("OutOfMemoryError: " + e.getMessage()); }
try { throw new AssertionError("explicit AssertionError is always thrown"); }
catch (Error e) { System.out.println(e.getClass().getSimpleName() + ": " + e.getMessage()); }
Thread t = new Thread(() -> { throw new Error("thread dies, JVM lives"); });
t.setUncaughtExceptionHandler((th, ex) -> System.out.println("handler got: " + ex.getMessage()));
t.start(); t.join();
System.out.println("JVM still running");
```

▶ Output

```
caught StackOverflowError, thread continues, depth > 1000: true
OutOfMemoryError: Requested array size exceeds VM limit
AssertionError: explicit AssertionError is always thrown
handler got: thread dies, JVM lives
JVM still running
```

💡 Pro tip

The message `Requested array size exceeds VM limit` means **one** allocation was too big (a bug), not that the heap is full.

📏 **Q692** EX-013 • Intermediate

How deep can recursion go before StackOverflowError, and how do you avoid it?

⚡ Quick answer

Each call pushes a frame onto the thread's stack (default **1 MB** for platform threads on 64-bit Linux, `-Xss` changes it). With simple one-argument frames the default allowed about **23,600** calls on JDK 25; `-Xss512k` halved it to 10,196; a 16 MB thread reached 416,824. Frame size matters: more locals and arguments mean fewer frames. Fix deep recursion with **iteration**, an **explicit stack**, or restructuring the algorithm, not a larger `-Xss`.

Measured on JDK 25 (these exact numbers vary with JIT state, frame size and platform):

Thread stack	Depth reached
512 KB (`-Xss512k`)	10,196
1 MB (default)	23,608

© TechTribers. For personal use only. Do not redistribute.

16 MB	416,824

Typical real causes: unbounded recursion (a missing base case), cyclic data structures traversed recursively (a bidirectional JPA relationship serialised to JSON), `toString` / `equals` / `hashCode` calling each other across a cycle, and deep trees or linked lists processed recursively. Java does **not** do tail-call optimisation. **Virtual threads** have growable stacks, so the limit is far higher, but a runaway recursion still ends in `StackOverflowError`.

Java

```
static int depth;
static void rec() { depth++; rec(); }
static int measure(long stackBytes) throws Exception {
    int[] res = new int[1];
    Thread t = new Thread(null, () -> { depth = 0; try { rec(); } catch (StackOverflowError e) {
res[0] = depth; } }, "probe", stackBytes);
    t.start(); t.join();
    return res[0];
}
int small = measure(512 * 1024), medium = measure(1024 * 1024), large = measure(16 * 1024 * 1024);
System.out.println((small < medium) + " " + (medium < large) + " " + (large > 100_000));
// converting recursion to an explicit stack: depth is limited by the heap, not by the thread stack
static long sumTo(int n) { Deque<Integer> stack = new ArrayDeque<>(); long sum = 0; stack.push(n);
while (!stack.isEmpty()) { int k = stack.pop(); sum += k; if (k > 0) stack.push(k - 1); } return
sum; }
System.out.println(sumTo(1_000_000));
```

▶ Output

```
true true true
500000500000
```

Recursion depth before StackOverflowError (simple one-argument frames, JDK 25, measured)



Depth scales linearly with stack size and inversely with frame size: more locals or arguments = fewer frames.

Fix deep recursion with iteration or an explicit stack, not a bigger -Xss.

> 💡 **Pro tip**
>
> Interview answer to "how would you avoid it?": add or fix the base case, convert to a loop or explicit `Deque`, memoise or restructure, and as a last resort run the work on a thread created with a larger stack size.

© TechTribers. For personal use only. Do not redistribute.

Q693 EX-014 • Advanced

What happens when an exception is thrown in a static initialiser? (ExceptionInInitializerError and NoClassDefFoundError)

⚡ **Quick answer**

If a static initialiser or static field initialiser throws, the **first** access to the class fails with `ExceptionInInitializerError` (wrapping the original as its cause). The class is then marked **erroneous**: every later access throws `NoClassDefFoundError: Could not initialize class X`, and the real cause is only in the first error. This confuses people because the second error has no useful stack trace.

Debugging: when you see `NoClassDefFoundError: Could not initialize class ...`, search the logs for the **earlier** `ExceptionInInitializerError` (or, from Java 17, the `Caused by` chain attached to the second error). Prevention: keep static initialisers trivial; load configuration lazily (holder class, `Supplier`, dependency injection); never perform I/O or call code that can throw checked-by-convention failures in a static block; and avoid cyclic static initialisation between classes.

An enum or singleton whose constructor throws fails the same way, which is why a failed Spring bean or JNDI lookup in a static field can take a whole class down for the life of the JVM.

Java

```
static class Config {
    static final int VALUE = compute();
    static int compute() { throw new IllegalStateException("config missing"); }
}
try { System.out.println(Config.VALUE); }
catch (ExceptionInInitializerError e) { System.out.println("1st access: " +
e.getClass().getSimpleName() + ", cause = " + e.getCause()); }
try { System.out.println(Config.VALUE); }
catch (NoClassDefFoundError e) { System.out.println("2nd access: " + e.getClass().getSimpleName() +
" - " + e.getMessage().replace("Main$", "")); }
try { new Config(); } catch (NoClassDefFoundError e) { System.out.println("any use of the class
keeps failing"); }
System.out.println(Config.class.getSimpleName() + " is still loaded: " + (Config.class != null));
```

▶ **Output**

```
1st access: ExceptionInInitializerError, cause = java.lang.IllegalStateException: config missing
2nd access: NoClassDefFoundError - Could not initialize class Config
any use of the class keeps failing
Config is still loaded: true
```

⚠️ **Interview trap**

A checked exception can't be thrown from a static initialiser (the compiler rejects it), so people wrap it in a `RuntimeException`: that wrapper is what ends up as the cause of the `ExceptionInInitializerError`.

© TechTribers. For personal use only. Do not redistribute.

Q694 EX-015

• Advanced

What happens when a constructor throws, and how do you avoid leaking resources and half-built objects?

⚡ Quick answer

If a constructor throws, `new` produces **no reference**: the caller never gets the object, and the partially constructed instance becomes garbage. But anything the constructor had **already acquired** (opened files, started threads, registered listeners, `this` published somewhere) stays acquired. Acquire resources **last**, or wrap the risky part in `try` / `catch` and release in the catch, then rethrow. Never let `this` escape from a constructor.

The first case leaks one handle (`open` stays 1): the object that counted it is unreachable, yet nothing released it. The second shows the repair: the constructor undoes its own acquisition before rethrowing, so the count returns to where it was.

Better designs: **two-phase** (a cheap constructor, an explicit `open()` / `start()`), a **static factory** that validates first and constructs last, or injecting already-opened resources. Also dangerous: starting a thread or registering `this` as a listener inside a constructor (another thread can see a half-built object), and calling overridable methods from constructors (the subclass isn't initialised yet; see the OOP part).

Java

```
static class Resource {
    static int open = 0;
    final String name;
    Resource(String name, boolean fail) { this.name = name; open++; if (fail) throw new
IllegalStateException("ctor failed after acquiring"); }
    void close() { open--; }
}
Resource r = null;
try { r = new Resource("a", true); }
catch (IllegalStateException e) { System.out.println("r is null: " + (r == null) + ", handles still
open: " + Resource.open); }
static class Safe {
    final Resource res;
    Safe(boolean fail) {
        Resource tmp = new Resource("b", false);
        try { if (fail) throw new IllegalStateException("late failure"); this.res = tmp; }
        catch (RuntimeException e) { tmp.close(); throw e; }                // release what this
constructor acquired
    }
}
try { new Safe(true); } catch (IllegalStateException e) { System.out.println("after cleanup, handles
open: " + Resource.open); }
```

▶ Output

```
r is null: true, handles still open: 1
after cleanup, handles open: 1
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

`finalize()` is not a safety net: it is deprecated for removal and can resurrect half-built objects. Use `Cleaner` or try-with-resources instead.

🕵️ **Q695** EX-016 • Advanced

How does "sneaky throw" work: why can generics throw a checked exception without declaring it?

⚡ **Quick answer**

Checked exceptions are enforced **only by the compiler**; the JVM treats all `Throwable` s alike. A generic method `<E extends Throwable> void sneaky(Throwable t) throws E { throw (E) t; }` is called with `E` inferred as `RuntimeException` , so the compiler thinks it throws only an unchecked type, while **type erasure** makes the cast a no-op and the real (checked) exception is thrown. Lombok's `@SneakyThrows` and some libraries use this.

Why it compiles: the call site is `sneaky<RuntimeException>` , which the compiler treats as throwing an unchecked exception. At runtime the bytecode has no `E` ; the thrown object is the `IOException` . The downside is the point of the question: the **caller cannot catch** `IOException` by name (the compiler rejects `catch (IOException)` around code that "can't" throw it), so such exceptions surface as surprises. Use it sparingly, in framework code, never to "avoid" a design decision.

Java

```
@SuppressWarnings("unchecked")
static <E extends Throwable> void sneaky(Throwable t) throws E { throw (E) t; }
static void unsafe() { Main.<RuntimeException>sneaky(new IOException("a checked exception, thrown
without any throws clause")); }
try { unsafe(); }
catch (Exception e) { System.out.println(e.getClass().getSimpleName() + " | is RuntimeException: " +
(e instanceof RuntimeException)); }
// the same effect via Callable: a lambda can throw checked exceptions, so call() hides nothing
java.util.concurrent.Callable<String> c = () -> { throw new IOException("declared by
Callable.call()"); };
try { c.call(); } catch (Exception e) { System.out.println(e.getMessage()); }
System.out.println(Throwable.class.isAssignableFrom(IOException.class) + " " +
java.lang.reflect.Modifier.isAbstract(Exception.class.getModifiers()));
```

▶ **Output**

```
IOException | is RuntimeException: false
declared by Callable.call()
true false
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Sneaky throws break the contract that readers get from `throws` clauses. Prefer wrapping in an unchecked exception with the cause preserved.

🪄 **Q696** EX-017 • Intermediate

How do you handle checked exceptions inside lambdas and streams?

⚡ **Quick answer**

The standard functional interfaces (`Function` , `Consumer` , `Supplier` ...) don't declare checked exceptions, so a lambda body that throws one won't compile. Options: **try/catch inside the lambda** and wrap in an unchecked exception (`UncheckedIOException`); a small **adapter** that converts a throwing functional interface into a standard one; use `Callable` (its `call()` declares `Exception`); or avoid streams for the I/O-heavy part and use a loop.

Which to choose: for **all-or-nothing** processing, wrap and let the first failure abort the stream; for **partial success** (collect good results and report failures) use a plain loop or map each item to a `Result` / `Optional` (see the Result question); never swallow the exception inside the lambda. A stream is lazy, so the exception appears at the **terminal operation**, not where `map` is written. `UncheckedIOException` preserves the `IOException` as its cause for callers that need it.

Java

```
@FunctionalInterface interface ThrowingFunction<T, R> { R apply(T t) throws Exception; }
static <T, R> Function<T, R> wrap(ThrowingFunction<T, R> f) {
    return t -> { try { return f.apply(t); } catch (RuntimeException e) { throw e; } catch
(Exception e) { throw new RuntimeException(e); } };
}
static String read(String name) throws IOException { if (name.isEmpty()) throw new
IOException("empty name"); return name.toUpperCase(); }
System.out.println(Stream.of("a", "b").map(wrap(Main::read)).toList());
try { Stream.of("a", "").map(wrap(Main::read)).toList(); }
catch (RuntimeException e) { System.out.println(e.getCause().getClass().getSimpleName() + ": " +
e.getCause().getMessage()); }
List<String> ok = new ArrayList<>(), failed = new ArrayList<>();
for (String s : List.of("x", "", "y")) { try { ok.add(read(s)); } catch (IOException e) {
failed.add("'" + s + "': " + e.getMessage()); } }
System.out.println(ok + " " + failed);
Callable<String> c = () -> read("z");
System.out.println(c.call());
System.out.println(Stream.of("a", "").map(s -> { try { return read(s); } catch (IOException e) {
throw new UncheckedIOException(e); } }).limit(1).toList());
```

▶ **Output**

```
[A, B]
IOException: empty name
[X, Y] ['': empty name]
```


© TechTribers. For personal use only. Do not redistribute.

```
Z
[A]
```

> 💡 **Pro tip**
> Libraries (Vavr `Try`, jOOL `Unchecked`) provide ready-made wrappers, but the 8-line adapter above covers most needs without a dependency.

Q697 EX-018 • Intermediate

How do you design a custom exception? (fields, constructors, hierarchy, cheap vs informative)

> ⚡ **Quick answer**
> Extend `Exception` (recoverable, caller must decide) or `RuntimeException` (programming or unrecoverable error); name it for the **failure**, not the layer; provide the four conventional constructors where useful (no-arg, message, cause, message+cause); carry **structured data** in fields (ids, amounts, error codes) so callers can react without parsing the message; keep fields immutable; declare `serialVersionUID` if the exception may cross a serialization boundary; and consider a base class per module for catch-all handling.

Checklist: (1) **one reason to exist**: if callers would never handle it differently from `IllegalStateException`, don't create a class; (2) **no sensitive data** in messages (passwords, tokens, card numbers); (3) include enough context to diagnose (ids, values), but build the message once; (4) don't override `getMessage` to something expensive; (5) generics are not allowed on exception classes; (6) for REST APIs map exceptions to status codes in **one place** (a `@ControllerAdvice`), not by catching in each controller.

Checked or unchecked? Follow the rule of thumb in the design section: unchecked for programming errors and for failures callers can't meaningfully recover from; checked when the caller **can and should** react (retry, compensate, ask the user).

Java

```java
static class InsufficientFundsException extends Exception {
    private static final long serialVersionUID = 1L;
    private final String account; private final long shortfall;
    InsufficientFundsException(String account, long shortfall) { super("account " + account + " is
short by " + shortfall); this.account = account; this.shortfall = shortfall; }
    InsufficientFundsException(String account, long shortfall, Throwable cause) { this(account,
shortfall); initCause(cause); }
    String account() { return account; } long shortfall() { return shortfall; }
}
static class ValidationException extends RuntimeException {                    // expected,
frequent: cheap
    ValidationException(String m) { super(m, null, false, false); }
}
static void withdraw(long balance, long amount) throws InsufficientFundsException { if (amount >
balance) throw new InsufficientFundsException("ACC-1", amount - balance); }
try { withdraw(100, 250); }
```


© TechTribers. For personal use only. Do not redistribute.

```
catch (InsufficientFundsException e) { System.out.println(e.getMessage() + " | account=" +
e.account() + " shortfall=" + e.shortfall() + " | " + e.toString().replace("Main$", "")); }
ValidationException v = new ValidationException("email is invalid");
System.out.println(v.getMessage() + " frames=" + v.getStackTrace().length + " " + (v instanceof
RuntimeException));
```

▶ Output

```
account ACC-1 is short by 150 | account=ACC-1 shortfall=150 | InsufficientFundsException: account
ACC-1 is short by 150
email is invalid frames=0 true
```

💡 **Pro tip**

Put a `Throwable.toString()` check in a test: `class name: message` is what appears in logs, so make sure both read well.

Section 03

try-with-resources & AutoCloseable

Close order, suppressed exceptions, designing AutoCloseable types, multi-resource failures and which JDK types now participate.

Questions 698–702 · 5 questions



Q698 EX-019 • Basic

How does try-with-resources work, and in what order are resources closed?

⚡ **Quick answer**

Any object whose type implements `AutoCloseable` can be declared in the `try ( ... )` header. The compiler generates the `finally`-like cleanup: resources are closed **in reverse order of declaration**, **after** the body and **before** any `catch` or `finally` clause runs. A `null` resource is allowed (its `close` is skipped). Since Java 9 you can use an existing **effectively final variable** as a resource (`try (shared) { ... }`).

Equivalent hand-written code is long (nested `try` / `finally`, null checks, suppression handling), which is why try-with-resources eliminated a whole class of leak bugs. Remember the order: **body → close (reverse) → catch → finally**. Because `close()` runs before `catch`, the handler can assume resources are released.

Java

```
static class Res implements AutoCloseable {
    final String n;
```


© TechTribers. For personal use only. Do not redistribute.

```
    Res(String n) { this.n = n; System.out.println("open " + n); }
    void use() { System.out.println("use " + n); }
    @Override public void close() { System.out.println("close " + n); }
}
try (Res a = new Res("A"); Res b = new Res("B")) { a.use(); b.use(); System.out.println("body"); }
try (Res c = new Res("C")) { throw new IllegalStateException("body failed"); }
catch (IllegalStateException e) { System.out.println("caught " + e.getMessage() + " (the resource
was already closed)"); }
finally { System.out.println("finally runs last"); }
Res shared = new Res("D");
try (shared) { System.out.println("Java 9: an effectively final variable can be the resource"); }
try (Res none = null) { System.out.println("a null resource is allowed; close is skipped"); }
```

▶ Output

```
open A
open B
use A
use B
body
close B
close A
open C
close C
caught body failed (the resource was already closed)
finally runs last
open D
Java 9: an effectively final variable can be the resource
close D
a null resource is allowed; close is skipped
```

💡 **Pro tip**

Declare each resource separately in the header; don't build a chain like `new BufferedReader(new FileReader(f))` inside one resource, or a failure creating the outer wrapper leaks the inner stream (see the multi-resource question).

Q699 EX-020 • Advanced

How do suppressed exceptions work, and why is try-with-resources better than try/finally?

⚡ **Quick answer**

If the body throws **X** and `close()` then throws **Y**, try-with-resources keeps **X as the primary exception** and attaches Y with `X.addSuppressed(Y)` (readable with `getSuppressed()`). A hand-written `try { ... } finally { r.close(); }` lets Y **replace** X, so the real failure is lost. When no body exception exists, the first `close()` failure becomes primary and later ones are suppressed.

In the last case `B` is closed first (reverse order) and fails, so "close failed: B" is primary and the failure of `A` is suppressed. `printStackTrace` shows suppressed exceptions under `Suppressed:`; loggers (SLF4J/Logback, Log4j 2) print them too. You can add your own with `addSuppressed` (for example in a manual cleanup loop closing

© TechTribers. For personal use only. Do not redistribute.

several resources and collecting failures). Disabled by the 4-argument `Throwable` constructor with `enableSuppression = false`.

Java

```
static class Bad implements AutoCloseable {
    final String n; Bad(String n) { this.n = n; }
    @Override public void close() { throw new IllegalStateException("close failed: " + n); }
}
static void manual() { Bad b = new Bad("manual"); try { throw new RuntimeException("body failed"); }
finally { b.close(); } }
try { manual(); }
catch (RuntimeException e) { System.out.println("manual finally: " + e.getMessage() + " |
suppressed=" + e.getSuppressed().length); }
try (Bad b = new Bad("twr")) { throw new RuntimeException("body failed"); }
catch (RuntimeException e) { System.out.println("try-with-resources: " + e.getMessage() + " |
suppressed=" + e.getSuppressed().length + " -> " + e.getSuppressed()[0].getMessage()); }
try (Bad a = new Bad("A"); Bad b = new Bad("B")) { System.out.println("no body failure"); }
catch (RuntimeException e) { System.out.println("first close failure wins: " + e.getMessage() + " |
suppressed: " + Arrays.stream(e.getSuppressed()).map(Throwable::getMessage).toList()); }
```

▶ Output

```
manual finally: close failed: manual | suppressed=0
try-with-resources: body failed | suppressed=1 -> close failed: twr
no body failure
first close failure wins: close failed: B | suppressed: [close failed: A]
```

try (A a = open(); B b = open()) { body throws X } → close in REVERSE order



If close() also throws Y: X stays the primary exception, Y is attached with addSuppressed(Y)

Manual try/finally: the exception from close() in finally REPLACES X, and X is lost.

Read them with e.getSuppressed(); stack traces print them as Suppressed: ...

⚠ **Interview trap**

The first example is why "close in finally" is a bug even when written carefully: the exception callers see is the cleanup error, which is usually the less interesting one.

© TechTribers. For personal use only. Do not redistribute.

Q700 EX-021

• Intermediate

How do you design an AutoCloseable class? (idempotent close, Closeable vs AutoCloseable)

⚡ **Quick answer**

`AutoCloseable.close() throws Exception` is the general interface; `java.io.Closeable.close() throws IOException` is the I/O version and is documented as **idempotent** (a second call has no effect). Design rules: make `close()` **idempotent**, declare the **narrowest** exception (or none) so callers needn't catch `Exception`, release in reverse order of acquisition, don't throw from `close()` unless the failure matters, and make methods fail with `IllegalStateException` after close.

Because `AutoCloseable.close()` declares `throws Exception`, a try-with-resources over a plain `AutoCloseable` forces callers to handle `Exception`; your own class can **narrow** that (override `close()` with no `throws`, as `Conn` does) and the compiler will stop asking. `Closeable` extends `AutoCloseable`, so every `Closeable` works in try-with-resources.

Idempotency matters because the same resource can be closed twice: explicitly and again by try-with-resources, or by a wrapper stream closing the one it wraps.

Java

```
static class Conn implements Closeable {
    boolean closed; int closeCalls;
    @Override public void close() { closeCalls++; if (closed) return; closed = true; }      //
idempotent: the second call does nothing
    void query() { if (closed) throw new IllegalStateException("connection is closed"); }
}
Conn c = new Conn();
try (c) { c.query(); }
c.close(); c.close();
System.out.println(c.closeCalls + " " + c.closed);
try { c.query(); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }
AutoCloseable lambda = () -> System.out.println("lambda resource closed");
try (lambda) { System.out.println("an AutoCloseable can even be a lambda"); }
System.out.println(Closeable.class.getInterfaces()[0].getSimpleName() + " | AutoCloseable.close
throws " + AutoCloseable.class.getMethod("close").getExceptionTypes()[0].getSimpleName()
        + " | Closeable.close throws " + Closeable.class.getMethod("close").getExceptionTypes()
[0].getSimpleName());
```

▶ **Output**

```
3 true
connection is closed
an AutoCloseable can even be a lambda
lambda resource closed
AutoCloseable | AutoCloseable.close throws Exception | Closeable.close throws IOException
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> If closing can fail in a way the caller can recover from, throw a **specific** checked exception from your `close()`; if not, swallow-and-log inside `close()` and keep the signature clean.

🧵 Q701 EX-022 • Advanced

What happens with several resources when one fails to open, and where do classic leaks still occur?

> ⚡ **Quick answer**
> Resources are opened in order; if the Nth fails to open, the **N-1 already-opened resources are closed** (in reverse order) before the exception propagates. The classic leak is **not using try-with-resources for the inner object** when wrapping: `new BufferedReader(new FileReader(f))` as one expression leaks the `FileReader` if the outer constructor throws. Declare each resource separately.

The final `open handles: 1` is the leak from `leaky()` that nothing ever repaired; `safe()` leaves the count unchanged (`inner2` was opened and closed). The same applies to streams (`new ObjectOutputStream(new FileOutputStream(f))`), JDBC (`Connection` → `Statement` → `ResultSet`, close in reverse order) and threads or executors started in constructors.

What the leak looks like in production: `Too many open files`, exhausted connection pools, slow memory growth in native buffers. Tools: `lsof`, JFR file events, leak detectors in pools (HikariCP `leakDetectionThreshold`).

Java

```
static int open = 0;
static class R implements AutoCloseable {
    final String n;
    R(String n, boolean fail) { this.n = n; if (fail) throw new IllegalStateException(n + " failed
to open"); open++; System.out.println("opened " + n); }
    public void close() { open--; System.out.println("closed " + n); }
}
try (R a = new R("A", false); R b = new R("B", true); R c = new R("C", false)) {
System.out.println("never reached"); }
catch (IllegalStateException e) { System.out.println(e.getMessage() + "; open handles: " + open); }
static void leaky() { R inner = new R("inner", false); R wrapper = new R("wrapper", true); }
// wrapper fails: inner is never closed
try { leaky(); } catch (IllegalStateException e) { System.out.println("leaky(): " + e.getMessage() +
"; open handles: " + open); }
static void safe() { try (R inner = new R("inner2", false); R wrapper = new R("wrapper2", true)) { }
}
try { safe(); } catch (IllegalStateException e) { System.out.println("safe(): " + e.getMessage() +
"; open handles: " + open); }
```

▶ **Output**

```
opened A
closed A
B failed to open; open handles: 0
opened inner
```


© TechTribers. For personal use only. Do not redistribute.

```
leaky(): wrapper failed to open; open handles: 1
opened inner2
closed inner2
safe(): wrapper2 failed to open; open handles: 1
```

⚠️ **Interview trap**

A wrapper's `close()` normally closes the wrapped stream, so closing only the outer is usually enough once it exists. The risk is the window *before* the outer object exists.

Q702 EX-023 • Intermediate

Which JDK types are AutoCloseable (ExecutorService, HttpClient, Files.lines...), and which aren't?

⚡ **Quick answer**

Beyond the obvious I/O streams, readers, `Scanner` and JDBC objects, newer JDK types joined: `ExecutorService` (Java 19: `close()` waits for submitted tasks, then shuts down), `HttpClient` (Java 21), `Stream` (via `BaseStream`, important for `Files.lines`, `Files.walk`, `Files.list`, `DirectoryStream`), `ZipFile`, `FileChannel`, `Formatter`. `Thread`, `Timer` and `Process` are **not**. (`ForkJoinPool` is, as an `ExecutorService`, but closing the common pool is a no-op.)

`Files.lines`, `Files.list`, `Files.walk` and `Files.find` return streams that **hold an open file or directory handle**: always use them in try-with-resources. The last line (which opens a stream and never closes it) is exactly the mistake to avoid; it prints `false` and leaks a handle until GC.

`ExecutorService.close()` (Java 19+) makes `try (var pool = Executors.newVirtualThreadPerTaskExecutor())` the idiomatic way to run a batch of tasks and wait for them all. Before Java 19 you called `shutdown()` and `awaitTermination()` yourself.

Java

```java
for (Class<?> c : List.of(ExecutorService.class, java.net.http.HttpClient.class, Scanner.class,
Stream.class, java.util.zip.ZipFile.class, java.sql.Connection.class,
        Formatter.class, java.nio.channels.FileChannel.class, Thread.class, Timer.class,
ForkJoinPool.class, StringWriter.class, Process.class))
    System.out.println(c.getSimpleName() + " " + AutoCloseable.class.isAssignableFrom(c));
try (ExecutorService pool = Executors.newFixedThreadPool(2)) {
    pool.submit(() -> System.out.println("task ran before close() returned"));
}
try (Stream<String> lines = "a\nb\nc".lines()) { System.out.println(lines.count()); }
System.out.println(Files.lines(Path.of("/etc/hostname")).getClass().getSimpleName().isEmpty());
```

▶ **Output**

```
ExecutorService true
HttpClient true
Scanner true
Stream true
ZipFile true
```


© TechTribers. For personal use only. Do not redistribute.

```
Connection true
Formatter true
FileChannel true
Thread false
Timer false
ForkJoinPool true
StringWriter true
Process false
task ran before close() returned
3
false
```

💡 **Pro tip**

IDE inspections ("resource opened but not closed") catch most of these; turn them on and treat them as errors in code review.

Section 04

Design & Best Practices

Checked vs unchecked, which standard exception to throw, Result types, assertions, messages, retries, layer translation and the common anti-patterns.

Questions 703–710 · 8 questions



⚖️ **Q703** EX-024 • Intermediate

Checked or unchecked: what are the guidelines, and why do modern frameworks prefer unchecked?

⚡ **Quick answer**

Classic guidance (Bloch, *Effective Java*): use **checked** exceptions for conditions the caller **can reasonably recover from**, **unchecked** for programming errors (precondition violations) and unrecoverable failures. In practice, checked exceptions **leak implementation details** (`SQLException` in a service signature), break lambdas and streams, and make API evolution harder, so Spring, Hibernate and most modern libraries use **unchecked** exceptions and translate checked ones at the boundary. The JDK's own API pairs show the trade-off.

Decision guide:

Situation	Use
Caller violated a precondition (null, bad range, wrong state)	Unchecked (`IllegalArgumentException` , `IllegalStateException` , NPE)

© TechTribers. For personal use only. Do not redistribute.

Caller can recover and should be forced to think about it (file missing, remote timeout)	Checked, or an unchecked with a documented type
Failure in a lower layer that the caller can't handle	Wrap in an unchecked exception with the cause
"Not found" is a normal outcome	`Optional` or a result type, not an exception
Library API consumed by lambdas	Unchecked

The `URI` pair (`new URI` checked, `URI.create` unchecked) is the pattern: offer both when the same operation is called with trusted input (a constant) and untrusted input (user data).

Java

```
try { new java.net.URI("http://bad host"); }
catch (java.net.URISyntaxException e) { System.out.println("new URI(...) -> checked " +
e.getClass().getSimpleName()); }
try { java.net.URI.create("http://bad host"); }
catch (IllegalArgumentException e) { System.out.println("URI.create(...) -> unchecked " +
e.getClass().getSimpleName() + " wrapping " + e.getCause().getClass().getSimpleName()); }
try { Files.readString(Path.of("/definitely/missing")); }
catch (IOException e) { System.out.println("Files.readString -> " + e.getClass().getSimpleName()); }
System.out.println(Files.exists(Path.of("/definitely/missing")) + " (a query method: no exception at
all)");
try { Optional.empty().orElseThrow(() -> new NoSuchElementException("not found: id 7")); }
catch (NoSuchElementException e) { System.out.println(e.getMessage()); }
System.out.println(Stream.of("1", "x", "3").map(s -> { try { return
Optional.of(Integer.parseInt(s)); } catch (NumberFormatException e) { return Optional.
<Integer>empty(); } }).flatMap(Optional::stream).toList());
```

▶ Output

```
new URI(...) -> checked URISyntaxException
URI.create(...) -> unchecked IllegalArgumentException wrapping URISyntaxException
Files.readString -> NoSuchFileException
false (a query method: no exception at all)
not found: id 7
[1, 3]
```

💡 **Pro tip**

In interviews, give both sides: checked exceptions document failure modes in the type system; unchecked exceptions avoid boilerplate and leaky abstractions. Then say what *you* do: unchecked by default, checked when callers have a real recovery.

© TechTribers. For personal use only. Do not redistribute.

Q704 EX-025

• Basic

Which standard exception should you throw? (IllegalArgument, IllegalState, UnsupportedOperation, NullPointer, IndexOutOfBounds…)

⚡ **Quick answer**

`IllegalArgumentException` : an **argument** is invalid. `IllegalStateException` : the **object's state** doesn't allow the call (closed, not started). `NullPointerException` : a required argument is `null` (use `Objects.requireNonNull`). `IndexOutOfBoundsException` : an index is out of range (`Objects.checkIndex`). `UnsupportedOperationException` : the operation is intentionally not supported (read-only collections). `NoSuchElementException` : asked for an element that isn't there. `ConcurrentModificationException` : structural change during iteration.

Conventions worth stating in an interview: **validate arguments at the top** of public methods and fail fast; the message says *what was wrong and with which value*; `NullPointerException` for a null argument (the JDK's own convention, via `requireNonNull`) rather than `IllegalArgumentException` ; `IllegalStateException` for "you called me in the wrong order"; don't invent a custom exception when a standard one communicates the same thing. For *domain* failures (insufficient funds, order not found) define a custom type.

Java

```
static void age(int a) { if (a < 0) throw new IllegalArgumentException("age must be >= 0 but was " +
a); }
static void send(boolean open) { if (!open) throw new IllegalStateException("connection is closed");
}
static void name(String s) { Objects.requireNonNull(s, "name must not be null"); }
static Object item(List<String> l, int i) { return l.get(Objects.checkIndex(i, l.size())); }
static void readOnly() { throw new UnsupportedOperationException("read-only view"); }
static void first(List<String> l) { if (l.isEmpty()) throw new NoSuchElementException("no first
element"); }
Map<String, Runnable> calls = new LinkedHashMap<>();
calls.put("age(-1)", () -> age(-1)); calls.put("send(false)", () -> send(false));
calls.put("name(null)", () -> name(null));
calls.put("item(list, 5)", () -> item(List.of("a"), 5)); calls.put("readOnly()", () -> readOnly());
calls.put("first(empty)", () -> first(List.of()));
calls.put("List.of(1).add(2)", () -> List.of(1).add(2));
for (var e : calls.entrySet()) {
    try { e.getValue().run(); }
    catch (RuntimeException ex) { System.out.println(String.format("%-20s %-30s %s", e.getKey(),
ex.getClass().getSimpleName(), ex.getMessage())); }
}
```

▶ **Output**

```
age(-1)              IllegalArgumentException       age must be >= 0 but was -1
send(false)          IllegalStateException          connection is closed
name(null)           NullPointerException           name must not be null
item(list, 5)        IndexOutOfBoundsException      Index 5 out of bounds for length 1
readOnly()           UnsupportedOperationException  read-only view
first(empty)         NoSuchElementException         no first element
List.of(1).add(2)    UnsupportedOperationException  null
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Throwing `IllegalArgumentException` for something that is really state (calling `read()` on a closed stream) misleads callers: the argument was fine, the object wasn't.

Q705 EX-026 • Advanced

When should you return a Result type or Optional instead of throwing? (sealed Result with pattern matching)

⚡ **Quick answer**

Use exceptions for **exceptional** failures and a value for **expected** outcomes. `Optional<T>` models "maybe a value"; a **sealed** `Result<T>` (`Ok` | `Err`) carries a success value or a typed error and forces callers to handle both through exhaustive pattern matching (Java 21). It shines for validation, parsing and batch processing where failures are data, not emergencies, and avoids both the cost of exceptions and the invisibility of error return codes.

Why the sealed hierarchy is more than style: the compiler checks the `switch` is **exhaustive**, so adding a new variant (`Pending` , `Timeout`) is a compile error everywhere it isn't handled. A batch job can then process 10,000 records, **collect** the 37 failures with reasons and keep going, which is awkward with exceptions in streams.

Trade-offs: result types add ceremony for chained operations (add `map` / `flatMap` helpers), and mixing them with exceptions in one codebase needs a clear boundary rule (for example: domain operations return results; infrastructure failures throw).

Java

```
sealed interface Result<T> permits Ok, Err { }
record Ok<T>(T value) implements Result<T> { }
record Err<T>(String error) implements Result<T> { }
static Result<Integer> parse(String s) {
    try { return new Ok<>(Integer.parseInt(s)); }
    catch (NumberFormatException e) { return new Err<>("not a number: " + s); }
}
static String show(Result<Integer> r) { return switch (r) { case Ok<Integer> ok -> "ok " +
ok.value(); case Err<Integer> err -> "error " + err.error(); }; }
System.out.println(show(parse("42")) + " | " + show(parse("x")));
List<Result<Integer>> rs = Stream.of("1", "x", "3", "y").map(Main::parse).toList();
System.out.println(rs.stream().filter(r -> r instanceof Ok).count() + " ok, " + rs.stream().filter(r
-> r instanceof Err).count() + " errors");
System.out.println(rs.stream().flatMap(r -> r instanceof Ok<Integer> ok ? Stream.of(ok.value()) :
Stream.empty()).mapToInt(i -> i).sum());
System.out.println(rs.stream().filter(r -> r instanceof Err).map(r -> ((Err<Integer>)
r).error()).toList());
```

▶ **Output**

```
ok 42 | error not a number: x
2 ok, 2 errors
```


© TechTribers. For personal use only. Do not redistribute.

```
4
[not a number: x, not a number: y]
```

> 💡 **Pro tip**
> Rule of thumb: if the **caller is expected to branch** on the failure (validation error, "not found"), return it; if the caller can only **abort or retry**, throw.

✅ Q706 EX-027 • Intermediate

What are assertions, and how do they differ from exceptions? (assert, -ea, AssertionError)

> ⚡ **Quick answer**
> `assert condition : message;` checks an **internal invariant** and throws `AssertionError` when false, but **only if assertions are enabled** (`java -ea`); they are **disabled by default**, and disabled assertions cost nothing and evaluate nothing. Use them for conditions that *can't happen if the code is correct* (postconditions, impossible branches); use **exceptions** for validating arguments, user input and anything external, because those checks must run in production.

This program is run with `-ea`. **Without** it the first `try` prints `not reached with -ea` (the assertion is skipped), `calls[0]` stays 0, and `enabled` stays `false`: the classic **side-effect trap**: never put code that must run inside an `assert` expression.

Practical advice: enable `-ea` in **tests and CI** (Maven Surefire and Gradle do by default), leave it off in production unless you accept the overhead; don't catch `AssertionError` in application code; use `Objects.requireNonNull` / `IllegalArgumentException` for anything a caller can get wrong. Enable per package with `-ea:com.example...`.

Java

```
try { assert 1 + 1 == 3 : "math is broken"; System.out.println("not reached with -ea"); }
catch (AssertionError e) { System.out.println("AssertionError: " + e.getMessage()); }
int[] calls = {0};
assert ++calls[0] > 0;
System.out.println("side effect executed (only because -ea is on): " + (calls[0] == 1));
boolean enabled = false;
assert enabled = true;
System.out.println("assertions enabled: " + enabled + " | status for this class: " +
Main.class.desiredAssertionStatus());
static int sqrt(int n) {
    if (n < 0) throw new IllegalArgumentException("n must be >= 0");                    //
argument check: always an exception
    int r = (int) Math.sqrt(n);
    assert r * r <= n && (r + 1) * (r + 1) > n : "postcondition violated";                //
internal invariant: an assertion
    return r;
}
System.out.println(sqrt(17));
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
AssertionError: math is broken
side effect executed (only because -ea is on): true
assertions enabled: true | status for this class: true
4
```

⚠️ Interview trap

Assertions are for the **programmer's** assumptions, exceptions for the **caller's** mistakes. A public method that validates its parameter with `assert` silently accepts bad input in production.

💬 **Q707** EX-028 • Intermediate

What makes a good exception message, and how do getMessage, toString and getLocalizedMessage differ?

⚡ Quick answer

A good message states **what failed, with which values, and why** (`"order 42 not found for customer 7"`), without secrets. `getMessage()` returns the detail message (possibly `null`); `toString()` is `className: getLocalizedMessage()` (or just the class name when the message is null); `getLocalizedMessage()` defaults to `getMessage()` and exists so subclasses can return a locale-specific text. `new RuntimeException(cause)` sets the message to `cause.toString()` .

Principles: (1) include the **offending value** and the **expected range or rule**; (2) don't repeat the cause's message in the wrapper, add context instead; (3) keep it free of passwords, tokens and personal data (messages end up in logs and error responses); (4) for API clients return an **error code** plus a safe message and keep the detailed one in server logs; (5) don't build messages with expensive calls that execute even when the exception is caught and discarded.

`getLocalizedMessage` is rarely overridden in modern code; frameworks typically map an error code to a localised message (`MessageSource` in Spring) at the API boundary.

Java

```
Exception plain = new RuntimeException();
Exception withMsg = new IllegalStateException("account is frozen");
Exception wrapped = new RuntimeException(new IOException("disk full"));
System.out.println(plain.getMessage() + " | " + plain + " | " + withMsg + " | " +
wrapped.getMessage());
class Localized extends Exception {
    Localized(String m) { super(m); }
    @Override public String getLocalizedMessage() { return "[localised] " + getMessage(); }
}
Localized l = new Localized("limit reached");
System.out.println(l.getMessage() + " | " + l.getLocalizedMessage() + " | " +
l.toString().replace("Main$", ""));
String bad = "Error", good = String.format("cannot transfer %d from account %s: balance %d is too
low", 500, "ACC-1", 120);
System.out.println(bad + " vs " + good);
```


© TechTribers. For personal use only. Do not redistribute.

```
try { List.of(1, 2, 3).get(7); } catch (IndexOutOfBoundsException e) {
System.out.println(e.getMessage()); }
try { Objects.checkIndex(5, 3); } catch (IndexOutOfBoundsException e) {
System.out.println(e.getMessage()); }
```

▶ Output

```
null | java.lang.RuntimeException | java.lang.IllegalStateException: account is frozen |
java.io.IOException: disk full
limit reached | [localised] limit reached | Localized: [localised] limit reached
Error vs cannot transfer 500 from account ACC-1: balance 120 is too low
Index 7 out of bounds for length 3
Index 5 out of bounds for length 3
```

💡 **Pro tip**

When reviewing code, search for `throw new .*Exception("")` and `getMessage()` only: those are the exceptions you'll fight with at 3 a.m.

🔄 **Q708** EX-029 • Advanced

How do you design retries? (transient vs permanent failures, backoff, idempotency)

⚡ **Quick answer**

Retry only **transient** failures (timeouts, 503, connection resets, deadlocks), never **permanent** ones (validation errors, 404, authentication failures). Use **exponential backoff with jitter**, a **maximum attempts / total time** limit, and retry only **idempotent** operations (or make them idempotent with a request id). Classify exceptions explicitly (a `TransientException` base or a predicate) so the policy isn't scattered through the code.

Production versions add: **jitter** (random 0–100% of the delay, so thousands of clients don't retry in lockstep), a **circuit breaker** (stop calling a dead dependency), **timeouts** on each attempt, metrics and logging of every retry, and **idempotency keys** for writes. Don't retry inside every layer: three layers each retrying three times is 27 attempts. Libraries: Resilience4j, Spring Retry, failsafe.

Never retry on `InterruptedException` (restore the interrupt flag and stop), on `Error`s, or on exceptions you can't classify as transient.

Java

```
static class TransientException extends RuntimeException { TransientException(String m) { super(m);
} }
static class PermanentException extends RuntimeException { PermanentException(String m) { super(m);
} }
static <T> T retry(int maxAttempts, long baseDelayMs, Callable<T> task, List<String> log) throws
Exception {
    for (int attempt = 1; ; attempt++) {
        try { return task.call(); }
        catch (TransientException e) {
```


© TechTribers. For personal use only. Do not redistribute.

```
            if (attempt == maxAttempts) throw e;                                    // out of
attempts: surface the last failure
            long delay = baseDelayMs * (1L << (attempt - 1));                        // 100,
200, 400 ...
            log.add("attempt " + attempt + " failed (" + e.getMessage() + "), retry in " + delay + "
ms");   // a real version would sleep here (with jitter)
        }
    }
}
int[] calls = {0};
List<String> log = new ArrayList<>();
String ok = retry(4, 100, () -> { if (++calls[0] < 3) throw new TransientException("timeout #" +
calls[0]); return "done after " + calls[0]; }, log);
System.out.println(ok + " " + log);
try { retry(3, 100, () -> { throw new PermanentException("invalid input"); }, new ArrayList<>()); }
catch (PermanentException e) { System.out.println("permanent errors are not retried: " +
e.getMessage()); }
try { retry(2, 50, () -> { throw new TransientException("still down"); }, new ArrayList<>()); }
catch (TransientException e) { System.out.println("gave up: " + e.getMessage()); }
```

▶ Output

```
done after 3 [attempt 1 failed (timeout #1), retry in 100 ms, attempt 2 failed (timeout #2), retry
in 200 ms]
permanent errors are not retried: invalid input
gave up: still down
```

⚠️ **Interview trap**

Retrying a non-idempotent call (charge a card, send an email) after a timeout can duplicate it: the first request may have succeeded. Make it idempotent or don't retry.

🏛️ **Q709** EX-030 • Intermediate

How should exceptions be translated at layer boundaries (DAO → service → API)?

⚡ **Quick answer**

Each layer should expose exceptions that match **its own abstraction**: the data layer converts driver errors into a data-access exception, the service layer converts those into **domain exceptions** (`OrderNotFoundException`), and the web layer maps domain exceptions to **HTTP responses** (status, error code, safe message). Always keep the **original as the cause**, and translate in **one place** per layer (for the web layer a single exception handler, e.g. Spring's `@ControllerAdvice` / RFC 7807 `ProblemDetail`) rather than in every method.

Notes: the response contains `detail` (safe, user-facing) but the **root cause stays server-side** (shown here only to prove the chain survived; in a real API log it, don't return it). Map unknown exceptions to a generic 500 and log them with the trace and a **correlation id** that the client also receives. In Spring, `@ExceptionHandler` methods in a `@RestControllerAdvice` return `ProblemDetail` (`application/problem+json`), giving every endpoint the same error shape.

© TechTribers. For personal use only. Do not redistribute.

```java
static class DaoException extends RuntimeException { DaoException(String m, Throwable c) { super(m, c); } }
static class OrderNotFoundException extends RuntimeException {
    final long id;
    OrderNotFoundException(long id, Throwable c) { super("order " + id + " not found", c); this.id = id; }
}
static String dao(long id) { if (id == 7) return "order-7"; throw new DaoException("no row for id " + id, new NoSuchElementException("0 rows")); }
static String service(long id) { try { return dao(id); } catch (DaoException e) { throw new OrderNotFoundException(id, e); } }
static Map<String, Object> controller(long id) {
    try { return Map.of("status", 200, "body", service(id)); }
    catch (OrderNotFoundException e) { return Map.of("status", 404, "title", "Order not found", "detail", e.getMessage(), "rootCause", e.getCause().getCause().getMessage()); }
    catch (RuntimeException e) { return Map.of("status", 500, "title", "Internal error"); }
// never leak internals to the client
}
System.out.println(new TreeMap<>(controller(7)));
System.out.println(new TreeMap<>(controller(9)));
```

▶ Output

```
{body=order-7, status=200}
{detail=order 9 not found, rootCause=0 rows, status=404, title=Order not found}
```

💡 **Pro tip**

Test the mapping: one unit test per exception type asserting status code, error code and that no stack trace or SQL appears in the body.

Q710 EX-031 • Intermediate

What are the exception-handling anti-patterns, and how do you spot them?

⚡ **Quick answer**

Swallowing (`catch (Exception e) { }`), **log-and-throw** (the same failure logged at every layer), **catch-all** `catch (Exception)` that hides bugs such as `ClassCastException`, **returning** `null` to signal failure, **losing the cause** when wrapping, **using exceptions for control flow**, **throwing from** `finally`, and **overly generic** `throws Exception` signatures. Each makes failures harder to detect or diagnose.

Anti-pattern	Fix
Empty `catch`	Handle, rethrow with cause, or log with a comment explaining why ignoring is safe
Log and rethrow at every layer	Log **once**, where the exception is finally handled (usually the boundary)

© TechTribers. For personal use only. Do not redistribute.

`catch (Exception)` **in business code**	Catch specific types; leave unexpected ones to a top-level handler
Return `null` **/-1 for failure**	Throw, return `Optional`, or a result type
`throw new X(e.getMessage())`	`throw new X("context", e)`
`throws Exception` **everywhere**	Declare the specific checked types, or wrap into an unchecked
`e.printStackTrace()`	Use the logger: `log.error("message", e)`

The first line of output is the log-and-throw problem: **three** error entries for one failure, which makes dashboards and alerts triple-count and hides the real number of incidents.

Java

```
static void dao(List<String> log) { try { throw new IllegalStateException("db down"); } catch
(IllegalStateException e) { log.add("DAO error: " + e.getMessage()); throw e; } }
static void service(List<String> log) { try { dao(log); } catch (IllegalStateException e) {
log.add("service error: " + e.getMessage()); throw e; } }
List<String> log = new ArrayList<>();
try { service(log); } catch (IllegalStateException e) { log.add("controller error: " +
e.getMessage()); }
System.out.println(log.size() + " log entries for ONE failure: " + log);
static String swallowed() { try { return Integer.toString(Integer.parseInt("x")); } catch (Exception
e) { return null; } }
System.out.println(swallowed() + "  <- the caller cannot tell \"no value\" from \"something
failed\"");
static String catchAll() { try { Object o = "s"; Integer i = (Integer) o; return "no error"; } catch
(Exception e) { return "catch (Exception) also hid a bug: " + e.getClass().getSimpleName(); } }
System.out.println(catchAll());
try { throw new IOException("root problem"); }
catch (IOException e) { RuntimeException r = new RuntimeException("wrapped, cause forgotten");
System.out.println("cause lost: " + (r.getCause() == null)); }
```

▶ **Output**

```
3 log entries for ONE failure: [DAO error: db down, service error: db down, controller error: db
down]
null  <- the caller cannot tell "no value" from "something failed"
catch (Exception) also hid a bug: ClassCastException
cause lost: true
```

> 💡 **Pro tip**
>
> A static-analysis rule set (SonarQube, Error Prone, SpotBugs) flags nearly all of these; wire it into CI so reviewers don't have to.

© TechTribers. For personal use only. Do not redistribute.

Section 05

Runtime Exceptions in Practice

A catalogue of the common runtime exceptions and their causes, arithmetic and overflow, exceptions in asynchronous code, and how to test exception behaviour.

Questions 711–714 · 4 questions



Q711 EX-032

• Basic

Which runtime exceptions do you see most often, and what causes each?

⚡ **Quick answer**

`NullPointerException` (null dereference), `ArrayIndexOutOfBounds` / `StringIndexOutOfBounds` / `IndexOutOfBounds` (bad index), `ClassCastException` (wrong downcast), `ArithmeticException` (integer divide by zero), `NumberFormatException` (bad number text), `ConcurrentModificationException` (structural change while iterating), `UnsupportedOperationException` (immutable or read-only collection), `IllegalStateException` / `IllegalArgumentException` (state or argument misuse), `NegativeArraySizeException`, `ArrayStoreException`, `NoSuchElementException`, `InputMismatchException`, `DateTimeParseException`, `EmptyStackException`.

Each line above proves a cause by actually triggering the exception. Quick map from exception to the usual fix:

Exception	Typical fix
`NullPointerException`	Validate inputs, return empty collections or `Optional`, `Objects.requireNonNull`
`IndexOutOfBounds` family	Check bounds, loop with `i < length`, use enhanced `for`
`ClassCastException`	Check with `instanceof` (pattern matching), fix generic raw types
`ConcurrentModificationException`	`removeIf`, `Iterator.remove`, copy, concurrent collection
`UnsupportedOperationException`	Copy to a mutable collection before modifying (`new ArrayList<>(list)`)
`NumberFormatException`	Validate or catch at the boundary; trim input
`IllegalStateException` (iterator)	Call `next()` before `remove()`, once per element

Java

```
Map<String, Runnable> m = new LinkedHashMap<>();
m.put("NullPointerException", () -> { String s = null; s.length(); });
m.put("ArrayIndexOutOfBoundsException", () -> { int[] a = new int[2]; a[2] = 1; });
m.put("StringIndexOutOfBoundsException", () -> "abc".substring(5));
```


© TechTribers. For personal use only. Do not redistribute.

```
m.put("ClassCastException", () -> { Object o = "s"; Integer i = (Integer) o; });
m.put("ArithmeticException", () -> { int z = 0; System.out.println(1 / z); });
m.put("NumberFormatException", () -> Integer.parseInt("12x"));
m.put("NegativeArraySizeException", () -> { int n = -1; int[] a = new int[n]; });
m.put("ArrayStoreException", () -> { Object[] a = new String[1]; a[0] = 1; });
m.put("ConcurrentModificationException", () -> { List<Integer> l = new ArrayList<>(List.of(1, 2,
3)); for (Integer i : l) l.remove(i); });
m.put("UnsupportedOperationException", () -> List.of(1).add(2));
m.put("IllegalStateException", () -> { Iterator<Integer> it = new ArrayList<>
(List.of(1)).iterator(); it.remove(); });
m.put("NoSuchElementException", () -> new ArrayList<Integer>().iterator().next());
m.put("InputMismatchException", () -> new Scanner("abc").nextInt());
m.put("DateTimeParseException", () -> java.time.LocalDate.parse("2026-13-01"));
m.put("IllegalArgumentException", () -> new ArrayList<Integer>(-1));
m.put("EmptyStackException", () -> new Stack<Integer>().pop());
for (var e : m.entrySet()) {
    try { e.getValue().run(); System.out.println(e.getKey() + " NOT thrown"); }
    catch (RuntimeException ex) { System.out.println(String.format("%-33s -> %s", e.getKey(),
ex.getClass().getSimpleName().equals(e.getKey()) ? "thrown as expected" : "DIFFERENT: " +
ex.getClass().getSimpleName())); }
}
```

▶ Output

```
NullPointerException              -> thrown as expected
ArrayIndexOutOfBoundsException    -> thrown as expected
StringIndexOutOfBoundsException   -> thrown as expected
ClassCastException                -> thrown as expected
ArithmeticException               -> thrown as expected
NumberFormatException             -> thrown as expected
NegativeArraySizeException        -> thrown as expected
ArrayStoreException               -> thrown as expected
ConcurrentModificationException   -> thrown as expected
UnsupportedOperationException     -> thrown as expected
IllegalStateException             -> thrown as expected
NoSuchElementException            -> thrown as expected
InputMismatchException            -> thrown as expected
DateTimeParseException            -> thrown as expected
IllegalArgumentException          -> thrown as expected
EmptyStackException               -> thrown as expected
```

💡 **Pro tip**

NPE, ClassCast, IndexOutOfBounds and NumberFormat cover most production incidents. Know their usual root causes and you can triage a log in seconds.

© TechTribers. For personal use only. Do not redistribute.

Q712 EX-033

• Intermediate

When does Java throw ArithmeticException, and how do you detect overflow?

⚡ Quick answer

Only **integer** division or remainder by zero throws `ArithmeticException` (`/ by zero`); **floating-point** division by zero gives `Infinity` or `NaN` with no exception. Integer arithmetic **silently overflows** (wraps around), so use `Math.addExact`, `subtractExact`, `multiplyExact`, `incrementExact`, `negateExact` and `toIntExact`, which throw `ArithmeticException` on overflow. `BigDecimal.divide` throws when the quotient has no exact decimal expansion unless you give a scale and rounding mode.

The silent cases are the dangerous ones: `Integer.MIN_VALUE / -1` and `Math.abs(Integer.MIN_VALUE)` both overflow back to `MIN_VALUE`; narrowing a `double` to `int` **saturates** (`3.99e10` becomes `Integer.MAX_VALUE`) and `NaN` becomes 0, with no exception. Use `*Exact` methods wherever overflow would be a bug (money in cents, sizes, offsets, timestamps arithmetic), and `BigInteger` / `BigDecimal` when the range is unbounded.

Java

```
System.out.println(Integer.MAX_VALUE + 1);                                    // silent overflow
try { Math.addExact(Integer.MAX_VALUE, 1); } catch (ArithmeticException e) {
System.out.println(e.getMessage()); }
try { Math.multiplyExact(1L << 62, 2L); } catch (ArithmeticException e) {
System.out.println(e.getMessage()); }
try { Math.toIntExact(3_000_000_000L); } catch (ArithmeticException e) {
System.out.println(e.getMessage()); }
try { System.out.println(5 / (int) 0.0); } catch (ArithmeticException e) {
System.out.println(e.getMessage()); }
try { System.out.println(5 % 0); } catch (ArithmeticException e) { System.out.println("remainder: "
+ e.getMessage()); }
System.out.println(5.0 / 0 + " " + -5.0 / 0 + " " + 0.0 / 0 + " " + (0.0 / 0 == 0.0 / 0) + " " +
Double.isNaN(0.0 / 0));
try { new java.math.BigDecimal("1").divide(new java.math.BigDecimal("3")); } catch
(ArithmeticException e) { System.out.println(e.getMessage()); }
System.out.println(new java.math.BigDecimal("1").divide(new java.math.BigDecimal("3"), 5,
java.math.RoundingMode.HALF_UP));
System.out.println(Integer.MIN_VALUE / -1 + " " + Math.abs(Integer.MIN_VALUE) + " " + (int) 3.99e10
+ " " + (int) Double.NaN + " " + (long) -1e30);
try { Math.negateExact(Integer.MIN_VALUE); } catch (ArithmeticException e) {
System.out.println(e.getMessage()); }
```

▶ Output

```
-2147483648
integer overflow
long overflow
integer overflow
/ by zero
remainder: / by zero
Infinity -Infinity NaN false true
Non-terminating decimal expansion; no exact representable decimal result.
0.33333
-2147483648 -2147483648 2147483647 0 -9223372036854775808
integer overflow
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`(a + b) / 2` overflows for large `a`, `b` (the famous binary-search bug). Use `a + (b - a) / 2` or `(a + b) >>> 1`.

Q713 EX-034 • Advanced

What happens to exceptions in executor tasks, CompletableFuture and virtual threads?

⚡ **Quick answer**

An exception in a task submitted with `submit` is **captured in the** `Future` and surfaces only as `ExecutionException` from `get()` (nothing is logged if you never call it). With `execute` the thread's `UncaughtExceptionHandler` runs. A `CompletableFuture` stores the failure: `join()` throws `CompletionException`, `get()` throws `ExecutionException`, and `exceptionally` / `handle` receive the exception, often wrapped in `CompletionException` (use `getCause()`). Virtual threads behave like platform threads.

The common production bug: `executor.submit(() -> doWork())` with the returned `Future` **ignored**: failures vanish silently. Either call `get()` (or `join()`), use `execute`, or wrap the task body in a try/catch that logs. For `CompletableFuture`, always end a chain with `exceptionally`, `handle` or `whenComplete`, otherwise an unobserved failure is lost.

Unwrapping helper: in `handle` / `exceptionally`, the exception may or may not be wrapped in `CompletionException` depending on **which stage** failed, so use `ex instanceof CompletionException && ex.getCause() != null ? ex.getCause() : ex`.

Java

```
ExecutorService pool = Executors.newFixedThreadPool(2);
Future<?> f = pool.submit(() -> { throw new IllegalStateException("task failed"); });
Thread.sleep(100);
System.out.println("submit(): nothing printed and nothing thrown until get() is called");
try { f.get(); } catch (ExecutionException e) { System.out.println("Future.get ->
ExecutionException, cause: " + e.getCause()); }
CompletableFuture<String> cf = CompletableFuture.supplyAsync(() -> { throw new
IllegalArgumentException("supplier failed"); });
try { cf.join(); } catch (CompletionException e) { System.out.println("join -> CompletionException,
cause: " + e.getCause().getClass().getSimpleName()); }
try { cf.get(); } catch (ExecutionException e) { System.out.println("get  -> ExecutionException,
cause: " + e.getCause().getClass().getSimpleName()); }
System.out.println(cf.exceptionally(ex -> "recovered from " + ex.getClass().getSimpleName() + "
wrapping " + ex.getCause().getClass().getSimpleName()).join() + " | completed exceptionally: " +
cf.isCompletedExceptionally());
System.out.println(CompletableFuture.supplyAsync(() -> 10).thenApply(x -> x / 0).handle((v, ex) ->
ex == null ? "value " + v : "handled " + ex.getCause().getClass().getSimpleName()).join());
Thread vt = Thread.ofVirtual().unstarted(() -> { throw new RuntimeException("virtual thread
failed"); });
vt.setUncaughtExceptionHandler((t, ex) -> System.out.println("handler: " + ex.getMessage()));
```


© TechTribers. For personal use only. Do not redistribute.

```
vt.start(); vt.join();
pool.shutdown();
```

▶ Output

```
submit(): nothing printed and nothing thrown until get() is called
Future.get -> ExecutionException, cause: java.lang.IllegalStateException: task failed
join -> CompletionException, cause: IllegalArgumentException
get  -> ExecutionException, cause: IllegalArgumentException
recovered from CompletionException wrapping IllegalArgumentException | completed exceptionally: true
handled ArithmeticException
handler: virtual thread failed
```

`executor.submit(task)` exception stored in the Future; nothing logged	future.get() → ExecutionException (cause = original)
`executor.execute(task)` thread dies; UncaughtExceptionHandler runs	stack trace on stderr by default
`CompletableFuture.supplyAsync` stored in the stage	join() → CompletionException · get() → ExecutionException
`exceptionally / handle` receive CompletionException wrapping the cause	unwrap with ex.getCause()
`virtual thread` same as a platform thread	setUncaughtExceptionHandler works too

⚠️ Interview trap

`ScheduledExecutorService.scheduleAtFixedRate` **cancels all future runs** if one execution throws, and nothing is logged. Catch inside the task.

🧪 Q714 EX-035 • Intermediate

How do you test exception behaviour (assertThrows) and what should the test check?

⚡ Quick answer

Assert that the code **throws the expected type** (and not a subtype by accident), and check the **message or error code**, the **cause** and any state that must be preserved. JUnit 5: `assertThrows(Type.class, () -> ...)` returns the exception for further assertions; AssertJ: `assertThatThrownBy(...).isInstanceOf(...).hasMessageContaining(...)`. Avoid `try { ...; fail(); } catch (...)` and `@Test(expected=...)` (JUnit 4), which can't check details.

What to assert, in order of value: (1) the **type**, (2) a stable part of the **message** or an error code (not the entire message string if it contains volatile data), (3) the **cause chain** when wrapping is part of the contract, (4) **no partial side effects** (balance unchanged, nothing written), (5) the happy path still works. Also test that **unexpected** inputs don't produce `NullPointerException` from deep inside (a sign of missing validation).

© TechTribers. For personal use only. Do not redistribute.

Java

```
static <T extends Throwable> T assertThrows(Class<T> type, Runnable code) {                // the
same contract as JUnit 5's assertThrows
    try { code.run(); }
    catch (Throwable t) { if (type.isInstance(t)) return type.cast(t); throw new
AssertionError("expected " + type.getSimpleName() + " but got " + t); }
    throw new AssertionError("expected " + type.getSimpleName() + " but nothing was thrown");
}
static void withdraw(long balance, long amount) { if (amount > balance) throw new
IllegalArgumentException("amount " + amount + " exceeds balance " + balance); }
IllegalArgumentException ex = assertThrows(IllegalArgumentException.class, () -> withdraw(100,
250));
System.out.println(ex.getMessage().contains("250") + " | " + ex.getMessage());
try { assertThrows(IllegalStateException.class, () -> withdraw(100, 250)); } catch (AssertionError
e) { System.out.println(e.getMessage()); }
try { assertThrows(RuntimeException.class, () -> withdraw(100, 50)); } catch (AssertionError e) {
System.out.println(e.getMessage()); }
// JUnit 5:
//   IllegalArgumentException ex = assertThrows(IllegalArgumentException.class, () ->
account.withdraw(250));
//   assertEquals("amount 250 exceeds balance 100", ex.getMessage());
//   assertEquals(100, account.balance());   // state unchanged after the failure
```

▶ Output

```
true | amount 250 exceeds balance 100
expected IllegalStateException but got java.lang.IllegalArgumentException: amount 250 exceeds
balance 100
expected RuntimeException but nothing was thrown
```

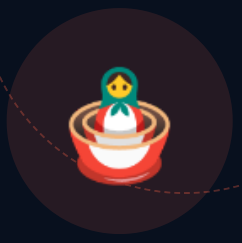
💡 **Pro tip**

Prefer asserting the exact exception **type** over `Exception.class`: `assertThrows(Exception.class, ...)` passes for any bug, including a typo that throws `NullPointerException`.

© TechTribers. For personal use only. Do not redistribute.

Section 06

Inner, Nested, Local & Anonymous Classes



What the compiler really generates, Outer.this and shadowing, memory retention (with measured GC evidence), captured variables, anonymous classes vs lambdas, and corner cases.

Questions 715–727 · 13 questions

Q715 EX-036

• Advanced

How does the compiler implement an inner class? (this$0, the hidden constructor parameter, and the JDK 18 change)

⚡ Quick answer

An inner (non-static) class is compiled to an ordinary class `Outer$Inner` whose constructor takes the **enclosing instance as a hidden first parameter**; `outer.new Inner()` passes it. If the inner class **uses** the outer instance, `javac` stores it in a synthetic field `this$0`. Since **JDK 18**, `javac` **omits that field** when the inner class doesn't use the outer instance (and isn't `Serializable`), so it no longer retains the outer object.

Everything the constructor line shows: `Outer$UsingInner(Outer)`: the source says `new UsingInner()` but the bytecode takes an `Outer`. That is why an inner class **can't be instantiated from a static context** without an outer instance (`new Inner()` in `static void main` is a compile error; write `outer.new Inner()`).

The JDK 18 change (JDK-8271623): previously **every** inner class kept `this$0` and could therefore leak its outer object (the textbook memory-leak warning). Now the field exists only when needed. The classic leak still occurs for inner classes, anonymous classes and lambdas that **do use** the outer instance (see the leak question), and for `Serializable` inner classes, which keep `this$0` to preserve the serialized form.

Java

```
static class Outer {
    int v = 7;
    class UnusedInner { }
    class UsingInner { int get() { return v; } }
    class SerInner implements java.io.Serializable { }
}
static String fields(Class<?> c) {
    StringBuilder sb = new StringBuilder();
    for (Field f : c.getDeclaredFields()) sb.append(f.getName()).append(f.isSynthetic() ? "
(synthetic)" : "").append(" ");
    return sb.toString().trim();
}
System.out.println("UnusedInner fields: [" + fields(Outer.UnusedInner.class) + "]");
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println("UsingInner fields:  [" + fields(Outer.UsingInner.class) + "]");
System.out.println("SerInner fields:    [" + fields(Outer.SerInner.class) + "]");
System.out.println(Outer.UsingInner.class.getDeclaredConstructors()
[0].toGenericString().replace("Main$", ""));
Outer o = new Outer();
Outer.UsingInner in = o.new UsingInner();
System.out.println(in.get() + " " + Modifier.isStatic(Outer.UsingInner.class.getModifiers()) + " " +
Outer.UsingInner.class.isMemberClass() + " " +
Outer.UsingInner.class.getEnclosingClass().getSimpleName());
```

▶ Output

```
UnusedInner fields: []
UsingInner fields:  [this$0 (synthetic)]
SerInner fields:    [this$0 (synthetic)]
Outer$UsingInner(Outer)
7 false true Outer
```



inner class that does NOT use the outer	**no this$0 field (JDK 18+)**
inner class that uses an outer field or method	**this$0 kept: outer stays reachable**
Serializable inner class	**this$0 kept (serialized form)**
anonymous class using an outer member	**this$0 kept**
lambda using this / a field	**captures this**
static nested class	**never has an outer reference**

Measured with GC: the outer object was collected for the first, anonymous-unused and lambda-unused cases; retained for the others.

💡 **Pro tip**

Whether `this$0` exists is visible with `javap -p -c Outer\$Inner` or the reflection above: a handy way to settle "does this inner class hold a reference to its parent?" arguments.

🔗 **Q716** EX-037 • Intermediate

How do Outer.this, outer.new Inner() and name shadowing work?

⚡ **Quick answer**

Inside an inner class, `this` is the inner instance and `Outer.this` is the enclosing instance. An inner instance is created with `outer.new Inner()` (or just `new Inner()` inside an instance method of `Outer`, which uses `this`). A field or parameter **shadows** an outer field of the same name; use `this.x` for the inner field and `Outer.this.x` for the outer one.

Resolution order for a bare name `name` inside `show`: **local variable or parameter** first, then a field of the **inner** class (and its superclasses), then a field of the **enclosing** class. The three prints are therefore `parameter`, `inner-`

© TechTribers. For personal use only. Do not redistribute.

`field`, `outer-field`. Two different `Outer` objects give two independent `Inner` families: an inner object is permanently bound to the outer instance it was created with.

```java
static class Outer {
    String name = "outer-field";
    class Inner {
        String name = "inner-field";
        void show(String name) { System.out.println(name + " | " + this.name + " | " +
Outer.this.name); }
        Outer outer() { return Outer.this; }
    }
    Inner make() { return new Inner(); }                    // inside an instance method: the
enclosing instance is 'this'
}
Outer o = new Outer();
Outer.Inner viaMethod = o.make();
Outer.Inner viaSyntax = o.new Inner();                      // from outside: qualify with the
enclosing instance
viaMethod.show("parameter");
System.out.println((viaSyntax.outer() == o) + " " + (viaMethod.outer() == o) + " " + (new
Outer().make().outer() == o));
```

▶ Output

```
parameter | inner-field | outer-field
true true false
```

⚠️ **Interview trap**

`outer.new Inner()` has the unusual syntax `instance.new Name()`. Interviewers like to ask why `Outer.Inner x = new Outer.Inner();` fails when `Inner` isn't static.

💧 Q717 EX-038 • Advanced

Do inner classes, anonymous classes and lambdas cause memory leaks? (measured)

⚡ **Quick answer**

Only when they **retain the enclosing instance**. Measured with a weak reference and GC on JDK 21 and 25: the outer object was collected for an inner class that doesn't use it, an anonymous class that doesn't use it and a lambda that doesn't use `this`; it was **retained** for an inner class that uses an outer member, a `Serializable` inner class, an anonymous class using an outer member and a lambda using `this` or a field. A short-lived inner object is harmless; a **long-lived** one (listener registry, cache, executor task) pins its whole outer object graph.

Where the real-world leaks come from: **Android-style handlers** and callbacks, **GUI listeners**, `ThreadLocal` values, event buses and caches holding lambdas (`map.put(key, () -> this.compute())`), and **thread pools** running anonymous `Runnable`s that capture a large request object. The fix is structural: use a **static nested class**

© TechTribers. For personal use only. Do not redistribute.

(or a lambda that captures only the needed values, not `this`), pass the data it needs explicitly, or hold the outer through a `WeakReference`.

How to prove a suspected leak: heap dump → find the instance → *Path to GC roots* in MAT; the path often shows `Outer$Inner.this$0` or `Outer$$Lambda.arg$1`.

Java

```
static class Outer implements Serializable {
    byte[] big = new byte[1_000_000]; int v = 7;
    class UnusedInner implements Runnable { public void run() { } }
    class UsingInner implements Runnable { public void run() { v++; } }
    class SerInner implements Runnable, Serializable { public void run() { } }
    Runnable anonUnused() { return new Runnable() { public void run() { } }; }
    Runnable anonUsing() { return new Runnable() { public void run() { v++; } }; }
    Runnable lambdaUnused() { return () -> { }; }
    Runnable lambdaUsing() { return () -> v++; }
    Runnable unusedInner() { return new UnusedInner(); } Runnable usingInner() { return new
UsingInner(); } Runnable serInner() { return new SerInner(); }
}
static boolean collected(Function<Outer, Runnable> f) throws Exception {
    Outer o = new Outer();
    Runnable r = f.apply(o);                                          // the callback we
keep, like a registered listener
    WeakReference<Outer> w = new WeakReference<>(o);
    o = null;
    for (int i = 0; i < 30 && w.get() != null; i++) { System.gc(); Thread.sleep(30); }
    boolean c = w.get() == null;
    r.run();                                                           // keeps r reachable
until here
    return c;
}
Map<String, Function<Outer, Runnable>> m = new LinkedHashMap<>();
m.put("inner class that does NOT use the outer", Outer::unusedInner); m.put("inner class that uses
an outer field", Outer::usingInner); m.put("Serializable inner class", Outer::serInner);
m.put("anonymous, unused outer", Outer::anonUnused); m.put("anonymous, uses outer field",
Outer::anonUsing); m.put("lambda, unused this", Outer::lambdaUnused); m.put("lambda, uses this",
Outer::lambdaUsing);
for (var e : m.entrySet()) System.out.println(String.format("%-42s outer collected after GC: %s",
e.getKey(), collected(e.getValue())));
```

▶ Output

```
inner class that does NOT use the outer    outer collected after GC: true
inner class that uses an outer field       outer collected after GC: false
Serializable inner class                   outer collected after GC: false
anonymous, unused outer                    outer collected after GC: true
anonymous, uses outer field                outer collected after GC: false
lambda, unused this                        outer collected after GC: true
lambda, uses this                          outer collected after GC: false
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Interview answer: "Inner classes can leak the outer instance if they use it and outlive it; since JDK 18 javac no longer retains it when unused. Static nested classes never can."

Q718 EX-039

• Intermediate

Static nested class vs inner class: when do you choose which, and what can each access?

> ⚡ **Quick answer**
> A **static nested class** is just a top-level class scoped inside another: no enclosing instance, so use it for helpers, builders, nodes and anything that doesn't need `this` of the outer; it can still access the outer's **private** members through a reference. An **inner class** has an enclosing instance and can use the outer's members directly: use it when the object is conceptually *part of* one outer instance (an iterator over *this* collection). Default to **static nested**; make it inner only when you need the outer instance.

	Static nested	Inner
Needs an outer instance	No	Yes (`outer.new Inner()`)
Access to outer instance members	Only via a reference	Directly
Can have static members	Yes	Yes since Java 16 (before: constants only)
Retains outer instance	Never	When it uses it (JDK 18+)
Typical use	`Builder` , `Node` , `Entry` , `Holder`	`Iterator` of a collection, event handler of one object

Private access works in both directions because since **Java 11 nest-based access control** the outer class and all its nested classes are **nestmates** (`getNestHost` , `isNestmateOf` above); earlier compilers generated synthetic `access$000` bridge methods.

Java

```
static class Outer {
    private int secret = 42;
    private static int counter = 0;
    class Inner { int peek() { return secret + counter; } }                    // needs an Outer
instance; uses its members directly
    static class Nested { int peek(Outer o) { return o.secret + counter; } }     // no outer
instance; reaches private members through a reference
    private static class Helper { }                                             // a private nested
class: an implementation detail
}
Outer o = new Outer();
Outer.Inner i = o.new Inner();
Outer.Nested n = new Outer.Nested();
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(i.peek() + " " + n.peek(o));
System.out.println(i.getClass().getNestHost().getSimpleName() + " " +
n.getClass().isNestmateOf(Outer.class) + " " + Modifier.isPrivate(Outer.Helper.class.getModifiers())
+ " " + Modifier.isStatic(Outer.Nested.class.getModifiers()) + " " +
Modifier.isStatic(Outer.Inner.class.getModifiers()));
```

▶ Output

```
42 42
Main true true true false
```

💡 **Pro tip**

`java.util.HashMap.Node` and `ArrayList.Itr` : the JDK uses a static nested class for data (`Node`) and an inner class where the iterator genuinely needs the list (`Itr`).

📌 **Q719** EX-040 • Intermediate

How do local classes capture variables, and why must they be effectively final?

⚡ **Quick answer**

A local class (declared inside a method) or an anonymous class can read the method's **local variables and parameters**, but only if they are **final or effectively final** (never reassigned). The reason is that the compiler **copies the value** into a synthetic field (`val$base`) of the class instance when it is created; if the variable could change afterwards, the copy and the original would silently diverge. Lambdas follow the same rule.

Local classes can also be generic-free helpers inside a method, implement interfaces, and since Java 16 be **records, enums and interfaces** too. They are **not** members (no access modifier, not visible outside the method). Captured variables increase the object's size by one field each; a local class that captures nothing is just a named helper.

The compile error you get for `int n = 0; Runnable r = () -> n++;` is *"local variables referenced from a lambda expression must be final or effectively final"*; the fixes are the array box above, `AtomicInteger` / `AtomicReference`, or restructuring (a `for` loop over the values instead of a counter captured by a lambda).

Java

```
static Runnable makeCounterPrinter(int start) {
    int base = start * 10;                                          // effectively final:
never reassigned
    class Printer implements Runnable { public void run() { System.out.println("captured base=" +
base); } }
    return new Printer();
}
static String fields(Class<?> c) { StringBuilder sb = new StringBuilder(); for (Field f :
c.getDeclaredFields()) sb.append(f.getName()).append(f.isSynthetic() ? " (synthetic)" : "").append("
"); return sb.toString().trim(); }
Runnable r = makeCounterPrinter(4);
r.run();
```

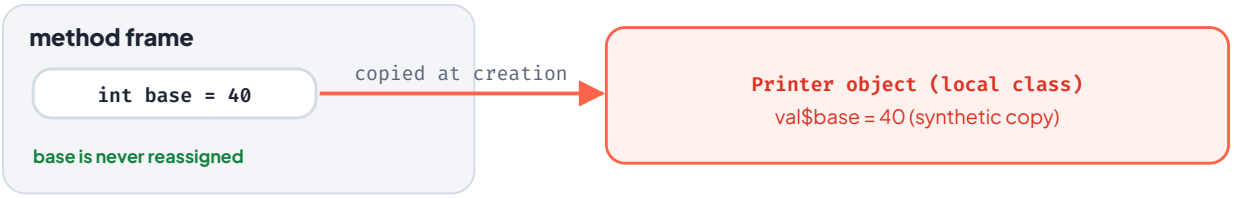

© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(fields(r.getClass()) + " | local=" + r.getClass().isLocalClass() + " | " +
r.getClass().getName().replace("Main$", "") + " | enclosing method: " +
r.getClass().getEnclosingMethod().getName());
int[] box = {0};
Runnable inc = () -> box[0]++;                                    // the REFERENCE is
effectively final; the contents may change
inc.run(); inc.run();
System.out.println(box[0] + " (a one-element array works around 'effectively final')");
AtomicInteger counter = new AtomicInteger();
Runnable inc2 = counter::incrementAndGet; inc2.run(); inc2.run(); inc2.run();
System.out.println(counter.get());
```

▶ Output

```
captured base=40
val$base (synthetic) | local=true | 1Printer | enclosing method: makeCounterPrinter
2 (a one-element array works around 'effectively final')
3
```

A local or anonymous class captures variables by COPY, which is why they must be effectively final



If the local could change after the object exists, the two copies would disagree. The compiler forbids it.

Workaround for mutable state: a one-element array or an AtomicInteger (the REFERENCE is final).

⚠️ **Interview trap**

Effectively final applies to the **variable**, not the object: you may call mutators on a captured `List`, but you may not reassign the variable.

Q720 EX-041 • Intermediate

How do anonymous classes work? (constructor arguments, initializer blocks, diamond, naming)

⚡ **Quick answer**

`new Type(args) { body }` declares and instantiates an **unnamed subclass** (or implementation) in one expression. Constructor arguments go to the **superclass constructor**; an **instance initializer block** `{ ... }` runs after it; the class has no name and no constructor of its own, so it can't be reused. The compiler names it `Outer$1`, `Outer$2` ... in textual order. Since Java 9 the **diamond** `new Comparator<>() { ... }` is allowed.

Limits and facts: an anonymous class can **extend a class or implement one interface, not both**; it can't declare constructors (use initializer blocks); it can have fields and extra methods (callable only inside it, or via `var`); and it

© TechTribers. For personal use only. Do not redistribute.

captures effectively final locals like any local class. The numbering `Outer$1` depends on source order, so inserting a new anonymous class earlier **renames** the later ones, which matters for serialization (see that section).

Use them when you need **state or multiple methods** in a one-off implementation; for a single-method interface prefer a **lambda**.

Java

```
static abstract class Greeter {
    final String greeting; int created;
    Greeter(String g) { greeting = g; }
    abstract String greet(String n);
}
Greeter g = new Greeter("Hello") {                                    // "Hello" goes to the
Greeter constructor
    { created = 99; }                                                 // instance initializer:
runs after the super constructor
    @Override String greet(String n) { return greeting + ", " + n + "!"; }
};
System.out.println(g.greet("Asha") + " " + g.created + " " + g.getClass().getName().replace("Main$",
"") + " " + g.getClass().isAnonymousClass() + " [" + g.getClass().getSimpleName() + "]");
Comparator<String> byLen = new Comparator<>() { public int compare(String a, String b) { return
Integer.compare(a.length(), b.length()); } };       // Java 9 diamond
System.out.println(byLen.compare("aa", "b") + " " + g.getClass().getSuperclass().getSimpleName() + "
" + g.getClass().getInterfaces().length);
Runnable r = new Runnable() {
    int runs;                                                         // an anonymous class may
have fields (a lambda cannot)
    public void run() { runs++; System.out.println("run #" + runs + " | simple name is empty: " +
getClass().getSimpleName().isEmpty()); }
};
r.run(); r.run();
```

▶ Output

```
Hello, Asha! 99 1 true []
1 Greeter 0
run #1 | simple name is empty: true
run #2 | simple name is empty: true
```

`Outer$Member`	member class	`isMemberClass() = true`	simple name: Member
`Outer$Nested`	static nested class	`isMemberClass() = true`	simple name: Nested
`Outer$1`	anonymous class	`isAnonymousClass() = true`	simple name: (empty)
`Outer$1Local`	local class	`isLocalClass() = true`	simple name: Local
`Outer$$Lambda/0x…`	lambda (hidden class)	`isSynthetic() = true`	created at run time

Numbers in $1, $2 follow textual order in the source file: reordering code changes class names (matters for serialization).

© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

The `var` trick: `var counter = new Object() { int n; };` lets you use the anonymous type's extra members: `counter.n++` compiles.

VS **Q721** EX-042 • Advanced

Lambda vs anonymous class: how do `this`, scope, class files and identity differ?

⚡ **Quick answer**

In a **lambda**, `this` is the **enclosing** instance; in an **anonymous class**, `this` is the **anonymous object** itself (the enclosing one is `Outer.this`). Lambdas don't introduce a new scope for names (a lambda parameter can't shadow a local), while an anonymous class does. Lambdas are compiled to a private method plus an `invokedynamic` call site that spins a **hidden class at run time** (`Main$$Lambda/0x...`), whereas an anonymous class is a normal class file `Outer$1.class` loaded with the application.

	Lambda	Anonymous class
`this`	The enclosing instance	The anonymous object
Applicable to	Functional interfaces only	Any class or interface
State (fields)	No	Yes
Class file	None (hidden class at run time)	`Outer$1.class`
Identity	No guarantees (the JVM may reuse instances)	New object each time
Startup cost	Bootstrap on first use, then cheap	Class load per class
Serializable	Only with an intersection cast / serializable interface	If the type is, and the enclosing `this$0` is too

Pitfall: a lambda that doesn't capture anything may be a **singleton** (non-capturing lambdas are cached), so never rely on `==` or `synchronized` on lambda instances.

Java

```
static class Widget {
    String name = "widget";
    Runnable asLambda() { return () -> System.out.println("lambda this -> " + this.name); }
    Runnable asAnonymous() { return new Runnable() { String name = "anon"; public void run() {
System.out.println("anonymous this -> " + this.name + ", outer -> " + Widget.this.name); } }; }
}
Widget w = new Widget();
w.asLambda().run();
w.asAnonymous().run();
Runnable l1 = () -> { }, l2 = () -> { };
Runnable a1 = new Runnable() { public void run() { } };
System.out.println(l1.getClass().isSynthetic() + " " + a1.getClass().isSynthetic() + " " +
```


© TechTribers. For personal use only. Do not redistribute.

```
l1.getClass().isAnonymousClass() + " " + a1.getClass().isAnonymousClass()
        + " " + l1.getClass().getName().startsWith("Main$$Lambda") + " " + (l1.getClass() ==
l2.getClass()));
int x = 10;
Runnable shadow = () -> { int y = x + 1; System.out.println("lambda sees x=" + x + ", y=" + y); };
shadow.run();
Runnable plain = () -> { };
Runnable serial = (Runnable & Serializable) () -> { };
System.out.println((plain instanceof Serializable) + " " + (serial instanceof Serializable));
```

▶ Output

```
lambda this -> widget
anonymous this -> anon, outer -> widget
true false false true true false
lambda sees x=10, y=11
false true
```

💡 **Pro tip**

When refactoring an anonymous class to a lambda, check the body for `this` : its meaning changes and it silently compiles.

Q722 EX-043 • Advanced

How do you extend an inner class from outside, and what is outer.super()?

⚡ **Quick answer**

A subclass of an inner class needs an **enclosing instance for the superclass constructor**. If the subclass is itself outside the outer class, its constructor must supply it with the special syntax `outer.super();` . If the subclass is also an inner class of the same outer, the enclosing instance is inherited implicitly. This is obscure but is a classic "can you extend an inner class?" question.

Without `outer.super()` the compiler reports *"an enclosing instance that contains Outer.Inner is required"*. Notice that `Ext` can read `Outer` 's `id` only through the inherited `Inner` behaviour: it has no `Outer.this` of its own. Design takeaway: inheritance from inner classes is awkward by nature; prefer **composition** or make the nested class `static` and pass what it needs explicitly.

Java

```
static class Outer {
    int id;
    Outer(int id) { this.id = id; }
    class Inner { String who() { return "Inner of " + id; } }
}
static class Ext extends Outer.Inner {                                  // a top-level
(here: static nested) subclass of an INNER class
    Ext(Outer outer) { outer.super(); }                                 // supplies the
enclosing instance to Inner's constructor
    @Override String who() { return "Ext/" + super.who(); }
}
```


© TechTribers. For personal use only. Do not redistribute.

```
Outer o = new Outer(5);
System.out.println(new Ext(o).who());
Outer.Inner anon = o.new Inner() { @Override String who() { return "anonymous subclass of " +
super.who(); } };
System.out.println(anon.who());
class LocalSub extends Outer.Inner { LocalSub(Outer out) { out.super(); } }
System.out.println(new LocalSub(new Outer(9)).who());
```

▶ Output

```
Ext/Inner of 5
anonymous subclass of Inner of 5
Inner of 9
```

⚠️ Interview trap

`o.new Inner() { ... }` (an anonymous subclass created with an explicit outer) is the legal way to subclass an inner class on the fly: no `super()` call needed.

NEW **Q723** EX-044 • Intermediate

Can inner classes have static members? What changed in Java 16 (and local records, enums, interfaces)?

⚡ Quick answer

Before Java 16, an inner class could declare only **constant** static fields (`static final` compile-time constants). **Java 16** (JEP 395, records) removed the restriction: inner classes may declare **static fields, methods and nested types**. Also since Java 16, **records, enums and interfaces can be declared locally** inside a method (they are implicitly static, so they can't capture locals).

Why records forced the change: a record is implicitly static, and records are allowed as members of inner classes, so the language had to permit static members there. Local records are popular for **intermediate data in stream pipelines** (`record Scored(String name, int score) {}` declared right before the stream). A local record or enum cannot reference the method's local variables or type parameters.

Java

```
static class Outer {
    class Inner {
        static int instances = 0;                                    // allowed since
Java 16
        static String describe() { return "static method in an inner class"; }
        Inner() { instances++; }
        record Point(int x, int y) { }                               // a nested
record inside an inner class
    }
}
Outer o = new Outer();
o.new Inner(); o.new Inner();
System.out.println(Outer.Inner.instances + " | " + Outer.Inner.describe() + " | " + new
Outer.Inner.Point(1, 2));
```


© TechTribers. For personal use only. Do not redistribute.

```
static String local() {
    record Pair(String a, int b) { }                    // local record
    enum Level { LOW, HIGH }                            // local enum
    interface Shape { double area(); }                  // local
interface
    Shape sq = () -> 4.0;
    return new Pair("p", 1) + " " + Level.HIGH + " " + sq.area();
}
System.out.println(local());
```

▶ Output

```
2 | static method in an inner class | Point[x=1, y=2]
Pair[a=p, b=1] HIGH 4.0
```

💡 **Pro tip**

Instead of `Map.Entry<String, Integer>` or `int[]` pairs in a stream, a tiny local record gives named fields and `equals` / `hashCode` / `toString` for free.

📦 **Q724** EX-045 • Intermediate

What are the implicit modifiers of nested types in interfaces and enums?

⚡ **Quick answer**

A member type declared inside an **interface** is implicitly `public static` (nested interfaces, enums and records are also static). Nested enums, records and interfaces are **implicitly static** anywhere. A class nested in an **enum** is a normal member. Enum constants with bodies are **anonymous subclasses** of the enum. These implicit rules show up in reflection and in questions about why `public` is optional.

The printed modifiers show `public static` for the class and interface inside `Api` even though the source says neither. Fields in an interface are `public static final`, methods `public abstract` (unless `default` / `static` / `private`). `Planet.MOON.getClass()` is an **anonymous subclass** (`Planet$2`), so use `getDeclaringClass()` to get the enum type, which is what `EnumMap` and `EnumSet` do internally.

© TechTribers. For personal use only. Do not redistribute.

```java
interface Api { class Defaults { } interface Callback { } enum Mode { ON, OFF } record Pair(int a, int b) { } int LIMIT = 10; }
enum Planet { EARTH { double g() { return 9.8; } }, MOON { double g() { return 1.6; } }; abstract double g(); static class Helper { } }
for (Class<?> c : List.of(Api.Defaults.class, Api.Callback.class, Api.Mode.class, Api.Pair.class, Planet.Helper.class))
    System.out.println(c.getSimpleName() + ": " + Modifier.toString(c.getModifiers()));
System.out.println(Planet.EARTH.g() + " " + Planet.MOON.getClass().getSuperclass().getSimpleName() + " " + Planet.MOON.getClass().isAnonymousClass() + " " +
Planet.MOON.getDeclaringClass().getSimpleName() + " " + Api.LIMIT);
System.out.println(Modifier.toString(Api.class.getField("LIMIT").getModifiers()));
```

▶ Output

```
Defaults: public static
Callback: public abstract static interface
Mode: public static final
Pair: public static final
Helper: static
9.8 Planet true Planet 10
public static final
```

⚠️ Interview trap

`Planet.MOON.getClass() == Planet.class` is **false** for constants with bodies. Use `getDeclaringClass()` or `==` on the constants themselves.

Q725 EX-046 • Intermediate

What is double-brace initialization, and why is it considered harmful?

⚡ Quick answer

`new HashMap<>() {{ put("a", 1); }}` is an **anonymous subclass** of `HashMap` with an **instance initializer block** (the second brace pair) that calls `put`. It's compact, but creates an extra **class per use**, the object's class is no longer `HashMap` (breaks `equals` implementations that compare `getClass()`), and in an instance method the anonymous class **captures** `this` (leak and serialization problems). Use `Map.of`, `List.of` or a builder instead.

The output shows each problem: the map's class is `Main$N` (an extra class loaded per use site), `getClass() == HashMap.class` is false, the instance has a synthetic `this$0` (it holds the `Factory`), and serialization fails because the hidden outer instance isn't serializable (a real-world issue when such maps end up in HTTP sessions or caches). `equals` between map implementations still works (it's defined by entries), but classes comparing `getClass()` don't.

Better: `Map.of(...)` / `Map.ofEntries` for small immutable maps, `new HashMap<>(Map.of(...))` for a mutable copy, a static factory or a builder, and a `static` initializer block for static maps.

© TechTribers. For personal use only. Do not redistribute.

```java
static class Factory {
    byte[] big = new byte[10];
    Map<String, Integer> make() { return new HashMap<>() {{ put("k", big.length); }}; }       //
uses an outer member: captures Factory.this
}
Map<String, Integer> m = new HashMap<>() {{ put("a", 1); put("b", 2); }};
System.out.println(m + " " + m.getClass().getName().replace("Main$", "") + " " + (m.getClass() ==
HashMap.class) + " " + m.equals(new HashMap<>(Map.of("a", 1, "b", 2))) + " " +
m.getClass().getSuperclass().getSimpleName());
Map<String, Integer> b = new Factory().make();
System.out.println(Arrays.stream(b.getClass().getDeclaredFields()).map(f -> f.getName()).toList() +
" " + b);
try { new ObjectOutputStream(new ByteArrayOutputStream()).writeObject(b); }
catch (NotSerializableException e) { System.out.println("not serializable: " +
e.getMessage().replace("Main$", "")); }
Map<String, Integer> immutable = Map.of("a", 1, "b", 2);
System.out.println(immutable.getClass().getSimpleName().startsWith("Map") + " " +
immutable.equals(m));
```

▶ Output

```
{a=1, b=2} 1 false true HashMap
[this$0] {k=10}
not serializable: Factory
true true
```

💡 Pro tip

Static analysis tools flag double-brace initialization (`SonarQube S3599`). If you see it in a review, replace it.

🧪 Q726 EX-047 • Advanced

How do generic inner classes work (Outer<T>.Inner) and should a collection's Node be inner or static?

⚡ Quick answer

An inner class of a generic class **shares the outer's type parameter**: `Stack2<T>.Node` can use `T` directly, and its full type is `Stack2<String>.Node` . A **static nested** class can't use the outer's `T` and needs its own parameter (`SNode<E>`). For data nodes in a collection, make them **static nested**: no outer reference and no need for the enclosing type parameter; use an inner class only when the node truly needs the collection instance.

This `Node` doesn't touch any outer member, so on JDK 18+ it has no `this$0` and costs nothing extra; on older compilers every node would have carried a hidden reference to the stack (4–8 bytes per node and an unintended retention). `java.util.LinkedList` uses `private static class Node<E>` for exactly this reason. Making the intent explicit with `static` is clearer than depending on a compiler optimisation.

© TechTribers. For personal use only. Do not redistribute.

```java
static class Stack2<T> {
    private class Node { T value; Node next; Node(T v, Node n) { value = v; next = n; } }
// inner: uses the outer's T
    private static class SNode<E> { E value; SNode<E> next; SNode(E v, SNode<E> n) { value = v; next
= n; } }  // static nested: its own type parameter
    private Node head;
    private SNode<T> shead;
    int size;
    void push(T v) { head = new Node(v, head); shead = new SNode<>(v, shead); size++; }
}
static String fields(Class<?> c) { StringBuilder sb = new StringBuilder(); for (Field f :
c.getDeclaredFields()) sb.append(f.getName()).append(f.isSynthetic() ? " (synthetic)" : "").append("
"); return sb.toString().trim(); }
Stack2<String> s = new Stack2<>();
s.push("a"); s.push("b");
Stack2<String>.Node n = s.head;
// the type of an inner class includes the outer's type arguments
System.out.println(n.value + " " + n.next.value + " " + s.shead.value + " " + s.size);
System.out.println("Node fields: [" + fields(Stack2.Node.class) + "] | SNode fields: [" +
fields(Stack2.SNode.class) + "]");
System.out.println(Stack2.Node.class.getDeclaredConstructors()[0].toGenericString().replace("Main$",
""));
```

▶ Output

```
b a b 2
Node fields: [value next] | SNode fields: [value next]
Stack2$Node(T,Stack2<T>$Node)
```

💡 **Pro tip**

If you can add `static` to a nested class without a compile error, you should. It documents that it needs no outer instance.

Q727 EX-048 • Intermediate

What are the names and flags of nested, local, anonymous and lambda classes (getName, getSimpleName, isMemberClass…)?

⚡ **Quick answer**

Member classes are named `Outer$Inner`; anonymous classes `Outer$1`, `Outer$2` (textual order); local classes `Outer$1Local`; lambdas become hidden classes `Outer$$Lambda/0x...`. `getSimpleName()` is the source name (empty for anonymous). `isMemberClass()`, `isLocalClass()`, `isAnonymousClass()`, `getEnclosingClass()`, `getEnclosingMethod()`, `getDeclaringClass()` and `getNestHost()` let you inspect the structure at run time.

Why this matters: **serialization** records the class name, so renaming or reordering anonymous classes changes `Outer$1` to `Outer$2` and breaks old data; **logging frameworks** use `getName()` for loggers; **proxies and**

© TechTribers. For personal use only. Do not redistribute.

frameworks skip synthetic classes; and `Outer.class.getDeclaredClasses()` returns **member** classes only (here 2), not local or anonymous ones. `getNestHost()` returns the top-level class that owns the whole nest (the harness class here, because the example classes are nested in it).

Java

```
static class Outer {
    class Member { }
    static class Nested { }
    Object anon() { return new Object() { }; }
    Object local() { class Local { } return new Local(); }
}
Outer o = new Outer();
Object[] objs = {o.new Member(), new Outer.Nested(), o.anon(), o.local()};
for (Object x : objs) {
    Class<?> c = x.getClass();
    System.out.println(c.getName().replace("Main$", "") + " simple=[" + c.getSimpleName() + "]
member=" + c.isMemberClass() + " local=" + c.isLocalClass() + " anon=" + c.isAnonymousClass()
            + " enclosing=" + (c.getEnclosingClass() == null ? null :
c.getEnclosingClass().getSimpleName()));
}
System.out.println(Outer.Member.class.getDeclaringClass().getSimpleName() + " " +
Outer.class.getDeclaredClasses().length + " " + Outer.Member.class.getNestHost().getSimpleName());
```

▶ Output

```
Outer$Member simple=[Member] member=true local=false anon=false enclosing=Outer
Outer$Nested simple=[Nested] member=true local=false anon=false enclosing=Outer
Outer$1 simple=[] member=false local=false anon=true enclosing=Outer
Outer$1Local simple=[Local] member=false local=true anon=false enclosing=Outer
Outer 2 Main
```

`Outer$Member`	member class	`isMemberClass() = true`	simple name: Member
`Outer$Nested`	static nested class	`isMemberClass() = true`	simple name: Nested
`Outer$1`	anonymous class	`isAnonymousClass() = true`	simple name: (empty)
`Outer$1Local`	local class	`isLocalClass() = true`	simple name: Local
`Outer$$Lambda/0x…`	lambda (hidden class)	`isSynthetic() = true`	created at run time

Numbers in $1, $2 follow textual order in the source file: reordering code changes class names (matters for serialization).

💡 **Pro tip**

`getSimpleName()` of an anonymous class is the empty string, a common cause of blank class names in logs. Use `getName()` there.

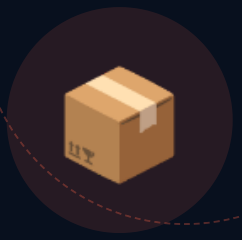
© TechTribers. For personal use only. Do not redistribute.

Section 07

Serialization in Depth

What really gets written, serialVersionUID in practice, evolving classes, custom serialization, proxies, Externalizable, object graphs, size, security and the traps.

Questions 728–739 · 12 questions



Q728 EX-049

• Intermediate

What exactly is serialized, and which constructors run on deserialization?

⚡ Quick answer

`ObjectOutputStream` writes the object's **non-static, non-transient instance fields**, plus those of every **serializable** superclass. On `readObject` the JVM allocates the object **without running its constructor** (or any serializable superclass constructor, or field initialisers); it only runs the **no-arg constructor of the first non-serializable superclass** to initialise that part. Static and transient fields are not written, and come back as `0` / `null` (or whatever that superclass constructor set).

Reading the result: `password` and `cache` (transient) are `null` / `0`; the **static** `count` is whatever the current JVM has (5, not the 77 at write time); `baseName` is back to `base-default` because `Base` isn't serializable (its field wasn't written, and its constructor re-initialised it); `Person(args)` did **not** run on deserialization, yet the field `derived` has its written value.

Consequences for design: (1) invariants enforced in constructors are **not** enforced on deserialization unless you re-validate in `readObject` / `readResolve`; (2) non-serializable superclasses need an **accessible no-arg constructor** (else `InvalidClassException: no valid constructor`); (3) `transient` fields must be recomputed; (4) `final` fields can be restored by serialization but not reassigned in `readObject`.

Java

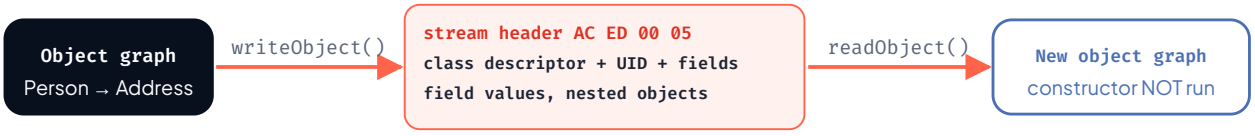
```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static class Base { String baseName = "base-default"; Base() { System.out.println("Base() runs:
first NON-serializable superclass"); } }
static class Person extends Base implements Serializable {
    String name; transient String password; static int count = 5; int age; transient int cache = 99;
String derived = "initialised";
    Person(String n, String p, int a) { System.out.println("Person(args) runs only for new"); name =
n; password = p; age = a; baseName = "base-changed"; }
}
Person p = new Person("asha", "s3cret", 30);
Person.count = 77;
```


© TechTribers. For personal use only. Do not redistribute.

```
byte[] bytes = ser(p);
Person.count = 5;
System.out.println("--- deserializing");
Person q = (Person) deser(bytes);
System.out.println(q.name + " | password=" + q.password + " | age=" + q.age + " | cache=" + q.cache
+ " | static count=" + Person.count + " | baseName=" + q.baseName + " | derived=" + q.derived);
```

▶ Output

```
Base() runs: first NON-serializable superclass
Person(args) runs only for new
--- deserializing
Base() runs: first NON-serializable superclass
asha | password=null | age=30 | cache=0 | static count=5 | baseName=base-default |
derived=initialised
```



On read: memory is allocated WITHOUT calling the class's constructor; only the first non-serializable superclass's no-arg constructor runs. transient and static fields are not written: they come back as 0 / null (or the superclass default).

Shared references and cycles are preserved inside one stream (identity is tracked by handles).

⚠️ **Interview trap**

A serializable class extending a non-serializable class **without a no-arg constructor** compiles fine and fails at runtime when read.

ID Q729 EX-050 • Intermediate

What does serialVersionUID do in practice? (a real InvalidClassException)

⚡ **Quick answer**

The stream stores the class's `serialVersionUID`; on read, the JVM compares it with the local class and throws `InvalidClassException` on mismatch. If you don't declare one, the JVM **computes** it from the class's name, interfaces, fields and methods, so almost **any** source change (even adding a method) changes it and breaks old data. Declaring `private static final long serialVersionUID = 1L;` makes compatibility **your decision**.

What gets the default UID wrong: **anything** in the class's shape. Adding a field, method or interface changes the computed value, so the first deployment after a harmless refactor throws `InvalidClassException: local class incompatible` on every old cached or persisted object. Declare the UID, **keep it** across compatible changes, and **change** it deliberately when the change is incompatible.

IDE and compiler support: IntelliJ and Eclipse can generate it, and `javac -Xlint:serial` warns when it is missing. `serialver Main\$Account` prints the computed value of a compiled class. Don't use a Java-serialized form for

© TechTribers. For personal use only. Do not redistribute.

long-term storage if you can avoid it (see the alternatives question).

```java
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static int indexOf(byte[] data, byte[] pat) { outer: for (int i = 0; i <= data.length - pat.length;
i++) { for (int j = 0; j < pat.length; j++) if (data[i + j] != pat[j]) continue outer; return i; }
return -1; }
static class Account implements Serializable { private static final long serialVersionUID = 1L;
String owner = "asha"; }
static class Implicit implements Serializable { String owner = "ravi"; }
byte[] bytes = ser(new Account());
byte[] name = "Main$Account".getBytes(StandardCharsets.UTF_8);
int pos = indexOf(bytes, name) + name.length;                                    // the 8 UID bytes
follow the class name in the stream
System.out.println("UID stored in the stream: " + java.nio.ByteBuffer.wrap(bytes, pos,
8).getLong());
bytes[pos + 7] = 2;                                                              // patch it:
simulates a class that changed incompatibly
try { deser(bytes); } catch (InvalidClassException e) {
System.out.println(e.getMessage().replace("Main$", "")); }
System.out.println(ObjectStreamClass.lookup(Account.class).getSerialVersionUID() + " explicit |
computed for Implicit is non-zero: " +
(ObjectStreamClass.lookup(Implicit.class).getSerialVersionUID() != 0));
```

▶ Output

```
UID stored in the stream: 1
Account; local class incompatible: stream classdesc serialVersionUID = 2, local class
serialVersionUID = 1
1 explicit | computed for Implicit is non-zero: true
```

The stream carries the writer's serialVersionUID; readObject compares it with the local class



Compatible (same UID kept)
add a field · remove a field · add or remove classes in the hierarchy (carefully) · change access modifiers · add writeObject

Incompatible (new UID needed)
change a field's type · change class hierarchy order · remove Serializable · switch to Externalizable · change an enum's constants meaning

Missing fields on read get the type's default (0, null): field initializers do not run, because no constructor is called.

💡 **Pro tip**

The `1L` convention is fine: the UID's job is to be **stable** and to change only on incompatible changes, not to be unique across classes.

© TechTribers. For personal use only. Do not redistribute.

🔄 Q730 EX-051 • Advanced

How do you evolve a serializable class? (compatible vs incompatible changes, missing fields)

⚡ Quick answer

With a **fixed serialVersionUID**: adding a field, removing a field, or adding/removing methods is **compatible**: on read, a field missing from the stream gets the type's **default** (`0` , `false` , `null`), and a stream field the class no longer has is ignored. **Incompatible**: changing a field's **type**, moving the class in the hierarchy, removing `Serializable` , or switching to `Externalizable` . Field **initialisers do not run** for old streams, so a new field is `null` , not its initial value.

The class-name patch is a laboratory trick (same-length names, same UID) to feed an "old" stream to a "new" class inside one JVM; in real life the old stream comes from a file, cache or message written by the previous release.

Strategies: (1) keep UIDs fixed and only make compatible changes; (2) give new fields **sensible defaults in** `readObject` (`if (email == null) email = ...`); (3) for incompatible changes, bump the UID and **migrate** data; (4) use `serialPersistentFields` and `GetField` for full control; (5) for anything stored long-term or exchanged between services, use a **schema-based format** (Avro, Protobuf, JSON) designed for evolution.

Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static int indexOf(byte[] data, byte[] pat) { outer: for (int i = 0; i <= data.length - pat.length;
i++) { for (int j = 0; j < pat.length; j++) if (data[i + j] != pat[j]) continue outer; return i; }
return -1; }
static class PersonV1 implements Serializable { private static final long serialVersionUID = 1L;
String name = "asha"; int age = 30; }
static class PersonV2 implements Serializable { private static final long serialVersionUID = 1L;
String name; int age; String email = "default@example.com"; }
static class PersonV3 implements Serializable { private static final long serialVersionUID = 1L;
String name; String age; }
byte[] v1 = ser(new PersonV1());                                    // an "old" stream,
written by version 1 of the class
byte[] asV2 = v1.clone();
byte[] asV3 = v1.clone();
byte[] oldName = "Main$PersonV1".getBytes(StandardCharsets.UTF_8);
asV2[indexOf(asV2, oldName) + oldName.length - 1] = (byte) '2';          // read the same bytes
as if the class had been evolved to V2
asV3[indexOf(asV3, oldName) + oldName.length - 1] = (byte) '3';          // ...or evolved
incompatibly (age: int -> String)
PersonV2 p2 = (PersonV2) deser(asV2);
System.out.println("compatible: " + p2.name + " " + p2.age + " email=" + p2.email + "   <- the new
field is null: its initialiser did not run");
try { deser(asV3); } catch (InvalidClassException e) { System.out.println("incompatible: " +
e.getMessage().replace("Main$", "")); }
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
compatible: asha 30 email=null   <- the new field is null: its initialiser did not run
incompatible: PersonV3; incompatible types for field age
```

⚠️ **Interview trap**

Adding `String email = "default"` and assuming old objects get that default is the classic mistake: they get `null` and the first `email.length()` throws.

🛠️ **Q731** EX-052 • Advanced

How does custom serialization work (writeObject, readObject, defaultWriteObject, validation)?

⚡ **Quick answer**

Define **private** `writeObject(ObjectOutputStream)` and `readObject(ObjectInputStream)` methods (the JVM finds them by exact signature via reflection). Call `defaultWriteObject()` / `defaultReadObject()` first to handle the normal fields, then write or read extra data in the **same order**. Typical uses: handling `transient` fields (encrypt, recompute), writing a compact form, and **validating invariants** on read (throw `InvalidObjectException`).

Rules that bite: the methods must be **private** with the exact signatures; read and write order must match; `readObject` runs on a **partially built** object, so don't call overridable methods; never leak `this` from it. The data in the stream can be crafted by an attacker, so `readObject` is a **trust boundary**: validate everything, copy mutable fields defensively (`Date`, arrays) after reading.

For classes whose serialized form should be completely independent of the implementation, prefer the **serialization proxy** pattern (next questions).

Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static class Secret implements Serializable {
    private static final long serialVersionUID = 1L;
    String user; transient String password;
    Secret(String u, String p) { user = u; password = p; }
    private void writeObject(ObjectOutputStream out) throws IOException {
        out.defaultWriteObject();                                    // writes 'user'
        out.writeObject(new StringBuilder(password).reverse().toString());  // "protect" the
transient field (a real system would encrypt)
    }
    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
        in.defaultReadObject();
        password = new StringBuilder((String) in.readObject()).reverse().toString();
        if (user == null || user.isBlank()) throw new InvalidObjectException("user must not be
blank");              // re-validate: constructors did not run
    }
```


© TechTribers. For personal use only. Do not redistribute.

```
}
byte[] b = ser(new Secret("asha", "s3cret"));
String raw = new String(b, StandardCharsets.ISO_8859_1);
System.out.println(raw.contains("s3cret") + " " + raw.contains("terc3s"));
Secret back = (Secret) deser(b);
System.out.println(back.user + " " + back.password);
try { deser(ser(new Secret(" ", "p"))); } catch (InvalidObjectException e) {
System.out.println("rejected: " + e.getMessage()); }
```

▶ Output

```
false true
asha s3cret
rejected: user must not be blank
```

💡 **Pro tip**

Evolving the custom format: write a **version number** first (`out.writeInt(2)`), then branch in `readObject` on it. It is the manual equivalent of a schema version.

Q732 EX-053 • Advanced

What are readResolve, writeReplace and the serialization proxy pattern?

⚡ **Quick answer**

`writeReplace()` lets an object substitute another object for itself **before writing**; `readResolve()` lets the deserialized object be **replaced by another** instance after reading. Together they fix two problems: **singletons and enums** (a deserialized copy would break identity: `readResolve` returns the existing instance) and **invariants** (the serialization proxy pattern writes a small private proxy whose `readResolve` rebuilds the real object through its **constructor**, so a forged stream cannot create an invalid instance).

Output line 1: `true false true` : the singleton with `readResolve` keeps identity, the one without gets a second instance, and **enums are always safe** (serialization writes the constant's name and resolves it on read), which is why the enum singleton is the recommended implementation.

The proxy pattern is *Effective Java* Item 90 ("consider serialization proxies instead of serialized instances"): the real class has no deserialization code path at all, the serialized form is decoupled from the class's fields (you can refactor freely), and records are the most compact way to write the proxy.

Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
enum Level { LOW, HIGH }
static class Singleton implements Serializable { static final Singleton INSTANCE = new Singleton();
private Singleton() { } private Object readResolve() { return INSTANCE; } }
static class BrokenSingleton implements Serializable { static final BrokenSingleton INSTANCE = new
BrokenSingleton(); private BrokenSingleton() { } }
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println((deser(ser(Singleton.INSTANCE)) == Singleton.INSTANCE) + " " +
(deser(ser(BrokenSingleton.INSTANCE)) == BrokenSingleton.INSTANCE) + " " + (deser(ser(Level.HIGH))
== Level.HIGH));
static final class Range implements Serializable {
    final int lo, hi;
    Range(int lo, int hi) { if (lo > hi) throw new IllegalArgumentException("lo > hi"); this.lo =
lo; this.hi = hi; }
    private Object writeReplace() { return new Proxy(lo, hi); }
// what is actually written
    private void readObject(ObjectInputStream in) throws InvalidObjectException { throw new
InvalidObjectException("proxy required"); }   // the real class is never read directly
    private record Proxy(int lo, int hi) implements Serializable { private Object readResolve() {
return new Range(lo, hi); } }
}
Range back = (Range) deser(ser(new Range(1, 5)));
System.out.println(back.getClass().getSimpleName() + " " + back.lo + ".." + back.hi);
try { deser(ser(new Range.Proxy(9, 1))); } catch (IllegalArgumentException e) {
System.out.println("forged stream rejected by the constructor: " + e.getMessage()); }
```

▶ Output

```
true false true
Range 1..5
forged stream rejected by the constructor: lo > hi
```



Serialization proxy pattern: the serialized form is a small private class; the real class refuses direct deserialization.

Every invariant is enforced by the ordinary constructor, so a forged stream cannot create an invalid object.

readResolve alone also fixes singletons: it replaces the deserialized copy with the existing instance.

💡 Pro tip

Records deserialize through the canonical constructor automatically, so a Serializable record is effectively a built-in serialization proxy (see the records question).

© TechTribers. For personal use only. Do not redistribute.

Q733 EX-054 • Intermediate

Externalizable vs Serializable: when would you use it?

⚡ Quick answer

`Externalizable` gives you **complete control**: you write and read every field yourself in `writeExternal` / `readExternal`, and the class must have a **public no-arg constructor** (which runs first on deserialization, unlike `Serializable`). The result can be smaller and faster because there's no reflection-driven field handling and no per-field metadata, but you take over versioning and correctness. It's rarely worth it today; prefer proxies, or a schema format for performance and compatibility.

Why the Externalizable stream is smaller: `Serializable` writes a **class descriptor for every field** (name, type signature) the first time a class appears in a stream; `Externalizable` writes only the class name and whatever bytes you write. The saving is a constant per stream, so it only matters for many small streams. Pitfalls: forgetting a field in both methods, mismatched order, no versioning, and subclass/superclass fields (you must handle the superclass state yourself).

Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static class Ext implements Externalizable {
    String a; int b;
    public Ext() { System.out.println("public no-arg constructor is REQUIRED and runs on read"); }
    Ext(String a, int b) { this.a = a; this.b = b; }
    @Override public void writeExternal(ObjectOutput out) throws IOException { out.writeUTF(a);
out.writeInt(b); }
    @Override public void readExternal(ObjectInput in) throws IOException { a = in.readUTF(); b =
in.readInt(); }
}
static class Std implements Serializable { String a; int b; Std(String a, int b) { this.a = a;
this.b = b; } }
System.out.println(ser(new Ext("hello", 42)).length + " bytes Externalizable vs " + ser(new
Std("hello", 42)).length + " bytes Serializable");
Ext back = (Ext) deser(ser(new Ext("hello", 42)));
System.out.println(back.a + " " + back.b);
```

▶ Output

```
43 bytes Externalizable vs 70 bytes Serializable
public no-arg constructor is REQUIRED and runs on read
hello 42
```

⚠️ Interview trap

The single most common runtime error with Externalizable is `InvalidClassException: no valid constructor`: the class lacks a **public** no-arg constructor.

© TechTribers. For personal use only. Do not redistribute.

Q734 EX-055 • Advanced

How does serialization handle shared references, cycles and repeated writes (reset, writeUnshared, deep copy)?

⚡ Quick answer

Within **one** `ObjectOutputStream`, each object is written **once** and later references are back-references (handles), so **shared references and cycles are preserved** on read. Writing the same object again (even after you changed it) writes only a handle, so the **stale state is sent**; `reset()` clears the handle table, `writeUnshared` writes a fresh copy. Serializing and deserializing an object is also a (slow) way to make a **deep copy**.

The last line shows the **stale-state trap**: `r3.name` is the *old* name (`n`) even though the object was modified before the third write, because the stream only wrote a handle; after `reset()` the new state (`changed`) goes out. This bites long-lived streams (sockets, RMI) that send the same mutable objects repeatedly: call `reset()` or use fresh objects.

Also: the handle table keeps **strong references** to every object ever written, so a long-lived `ObjectOutputStream` leaks memory unless you call `reset()` periodically.

Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static class Node implements Serializable { String name; Node next; Node(String n) { name = n; } }
static class Pair implements Serializable { Node left, right; }
Node shared = new Node("shared");
Pair p = new Pair(); p.left = shared; p.right = shared;
Pair q = (Pair) deser(ser(p));
System.out.println((q.left == q.right) + " " + (q.left != shared));                    // identity
preserved inside the graph; but these are new objects
Node a = new Node("a"), b = new Node("b"); a.next = b; b.next = a;
Node a2 = (Node) deser(ser(a));
System.out.println((a2.next.next == a2) + " " + a2.next.name);                         // a cycle is
handled
ByteArrayOutputStream bo = new ByteArrayOutputStream();
ObjectOutputStream out = new ObjectOutputStream(bo);
Node n = new Node("n");
out.writeObject(n); out.flush(); int first = bo.size();
out.writeObject(n); out.flush(); int second = bo.size() - first;                       // the same
object again: only a back-reference
n.name = "changed";
out.writeObject(n); out.flush(); int third = bo.size() - first - second;               // changed
state is NOT written: stale data
out.reset();
out.writeObject(n); out.flush(); int afterReset = bo.size() - first - second - third;  // after
reset() the object is written in full again
System.out.println(first + " " + second + " " + third + " " + afterReset);
ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(bo.toByteArray()));
Node r1 = (Node) in.readObject(), r2 = (Node) in.readObject(), r3 = (Node) in.readObject(), r4 =
```


© TechTribers. For personal use only. Do not redistribute.

```
(Node) in.readObject();
System.out.println((r1 == r2) + " " + r3.name + " " + r4.name);
```

▶ Output

```
true true
true b
84 5 5 87
true n changed
```

⚠️ Interview trap

A deep copy via serialization requires **everything** in the graph to be serializable and is orders of magnitude slower than a copy constructor. Use it in tests, not in hot paths.

Q735 EX-056 • Intermediate

How big and how fast is Java serialization, and what are the alternatives?

⚡ Quick answer

Java serialization is **verbose**: a two-int object takes **43 bytes** (the stream header `AC ED 00 05`, the class descriptor with the class name and field metadata, then the data), versus **8 bytes** for the same two ints written with `DataOutputStream` and 13 bytes as JSON text. It's also slow, version-fragile and a security liability. Alternatives: **JSON** (Jackson), **Protocol Buffers**, **Avro**, **Kryo/FST** (fast, Java-specific), **Cap'n Proto/FlatBuffers**, or hand-written binary formats.

Format	Pros	Cons
Java serialization	No code, handles graphs and cycles	Large, slow, fragile, unsafe on untrusted data
JSON (Jackson)	Readable, interoperable, tolerant to evolution	Text size, no cycles by default, slower than binary
Protobuf / Avro	Compact, schema-driven, forward/backward compatible	Needs schema and code generation
Kryo / FST	Very fast, compact, Java-specific	Class registration, version fragility
Hand-written `DataOutput`	Smallest and fastest	You own the format and versioning

Note `Integer[1000]` is 1,045 bytes while `int[1000]` is **4,027** (4,000 bytes of data plus 27 of metadata): the boxed array is smaller only because the test array is full of `null`s (each `null` takes one byte). For real data the boxed array is far larger. Where Java serialization is still unavoidable (HTTP session replication, some JMX/RMI), keep the classes small and stable, and treat the input as untrusted.

© TechTribers. For personal use only. Do not redistribute.

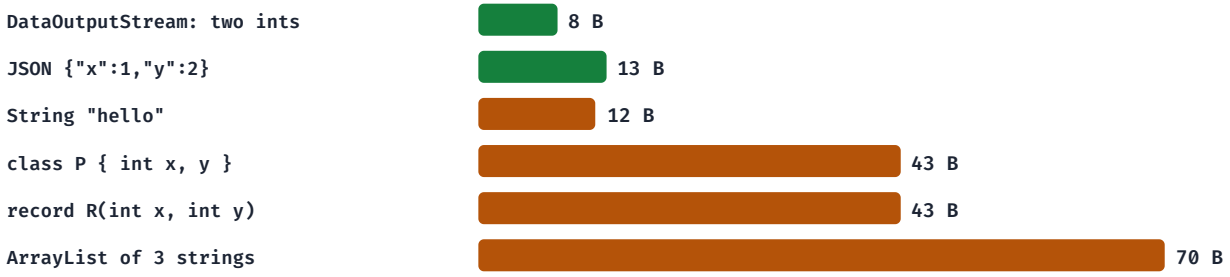
Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static class P implements Serializable { int x = 1, y = 2; }
record R(int x, int y) implements Serializable { }
byte[] javaSer = ser(new P()), recSer = ser(new R(1, 2));
ByteArrayOutputStream b = new ByteArrayOutputStream();
try (DataOutputStream d = new DataOutputStream(b)) { d.writeInt(1); d.writeInt(2); }
String json = "{\"x\":1,\"y\":2}";
System.out.println(javaSer.length + " " + recSer.length + " " + b.size() + " " +
json.getBytes(StandardCharsets.UTF_8).length);
System.out.println(ser(new Integer[1000]).length + " " + ser(new int[1000]).length + " " + ser(new
ArrayList<>(List.of("a", "b", "c"))).length + " " + ser("hello").length);
byte[] header = ser("x");
System.out.printf("stream header: %02x %02x %02x %02x%n", header[0], header[1], header[2],
header[3]);
```

▶ Output

```
43 43 8 13
1045 4027 70 12
stream header: ac ed 00 05
```

Bytes on the wire for tiny objects (measured): Java serialization pays for class metadata every stream



Integer[1000] (all null) = 1,045 B · int[1000] = 4,027 B · every stream starts AC ED 00 05

Java serialization is convenient but verbose, brittle across versions and unsafe on untrusted input.

💡 **Pro tip**

The OpenJDK team recommends **not** using Java serialization for new designs and is working toward deprecating it; a serialization-framework replacement is on the Valhalla/Amber roadmap.

© TechTribers. For personal use only. Do not redistribute.

Q736 EX-057 • Advanced

Why is deserialization dangerous, and how do you harden it? (code runs before the cast)

⚡ Quick answer

`readObject` **executes code from the stream** while reconstructing objects: class loading, `readObject` methods, `readResolve`, `hashCode` of map keys and so on. An attacker who controls the bytes can chain existing classes (**gadget chains**) into remote code execution *before* your cast or validation runs. Defences: **don't deserialize untrusted data**, prefer JSON/Protobuf, and if you must, use an **allow-list** `ObjectInputFilter` (JEP 290, process-wide with JEP 415), limit depth and size, and keep dependencies patched.

The first message is the key lesson: **type-checking the result is too late**. The filter rejects the class *before* instantiation (the second message), so its `readObject` never runs.

Hardening checklist: (1) avoid Java deserialization of network/user data entirely; (2) otherwise set an **allow-list filter** (`jdk.serialFilter` system property or `ObjectInputFilter.Config.setSerialFilter`) with `maxdepth`, `maxrefs`, `maxbytes`, `maxarray`; (3) use **context-specific filter factories** (JEP 290 / Java 17's `setSerialFilterFactory`); (4) keep libraries with known gadget chains (old Commons Collections, Spring, etc.) updated; (5) sign or MAC serialized data you must accept; (6) log rejected attempts. This pattern is behind many high-profile CVEs (Apache Commons Collections, WebLogic, Jenkins, Log4Shell-adjacent issues).

Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static class Evil implements Serializable {
    static boolean ran;
    private void readObject(ObjectInputStream in) throws Exception { in.defaultReadObject(); ran =
true; }          // an attacker-chosen gadget would run HERE
}
byte[] attack = ser(new Evil());
try { String s = (String) deser(attack); }
catch (ClassCastException e) { System.out.println("the cast failed, but readObject had already run:
" + Evil.ran); }
ObjectInputFilter allowList =
ObjectInputFilter.Config.createFilter("java.base/*;maxdepth=5;maxbytes=1000;!*");    // allow only
JDK base classes, reject everything else
Evil.ran = false;
try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(attack))) {
in.setObjectInputFilter(allowList); in.readObject(); }
catch (InvalidClassException e) { System.out.println("filter rejected it: " +
e.getMessage().replace("Main$", "") + " | readObject ran: " + Evil.ran); }
try (ObjectInputStream in = new ObjectInputStream(new ByteArrayInputStream(ser(new ArrayList<>
(List.of("ok")))))) { in.setObjectInputFilter(allowList); System.out.println("allowed: " +
in.readObject()); }
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
the cast failed, but readObject had already run: true
filter rejected it: filter status: REJECTED | readObject ran: false
allowed: [ok]
```

⚠️ **Interview trap**

A `ClassCastException` after `readObject` means the dangerous part already happened. Validate **before** deserializing (filter), not after.

Q737 EX-058 • Intermediate

How are records, enums and lambdas serialized?

⚡ **Quick answer**

Records serialize their component values and are rebuilt through the **canonical constructor** on read, so their validation runs and no `readObject` / `writeObject` hooks are used (a safe default). **Enums** serialize only the constant's **name** and resolve to the existing constant on read (identity preserved, field values not serialized). **Lambdas** are serializable only if the target type is `Serializable` (an **intersection cast** `(Runnable & Serializable)`), and their serialized form is tied to the enclosing class's **synthetic method names**, which makes it fragile.

The forged stream proves the record property: the canonical constructor ran with `x = -1` and rejected it, whereas a classic class with the same field would have accepted the forged value silently. For enums, never serialize by ordinal or rely on constant order: serialization stores the **name**, so renaming a constant breaks old data (`InvalidObjectException: No enum constant`). Serializing lambdas works but should be avoided: the format depends on the lambda's position in the source.

Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static int indexOf(byte[] data, byte[] pat) { outer: for (int i = 0; i <= data.length - pat.length;
i++) { for (int j = 0; j < pat.length; j++) if (data[i + j] != pat[j]) continue outer; return i; }
return -1; }
record Point(int x, int y) implements Serializable { Point { if (x < 0) throw new
IllegalArgumentException("x must be >= 0"); } }
enum Color { RED, GREEN }
System.out.println(deser(ser(new Point(1, 2))) + " " + (deser(ser(Color.GREEN)) == Color.GREEN));
byte[] bytes = ser(new Point(1, 2));
int at = indexOf(bytes, new byte[]{0, 0, 0, 1, 0, 0, 0, 2});                    // the two
ints are the last bytes of the stream
bytes[at] = (byte) 0xFF; bytes[at + 1] = (byte) 0xFF; bytes[at + 2] = (byte) 0xFF; bytes[at + 3] =
(byte) 0xFF;       // forge x = -1
try { deser(bytes); } catch (Exception e) { System.out.println(e.getClass().getSimpleName() + " <- "
+ (e.getCause() == null ? e.getMessage() : e.getCause().getClass().getSimpleName() + ": " +
e.getCause().getMessage())); }
Supplier<String> notSerializable = () -> "x";
```


© TechTribers. For personal use only. Do not redistribute.

```
try { ser(notSerializable); } catch (NotSerializableException e) { System.out.println("plain lambda:
NotSerializableException"); }
Supplier<String> lambda = (Supplier<String> & Serializable) () -> "from a serialized lambda";
System.out.println(((Supplier<String>) deser(ser(lambda))).get());
```

▶ Output

```
Point[x=1, y=2] true
InvalidObjectException <- IllegalArgumentException: x must be >= 0
plain lambda: NotSerializableException
from a serialized lambda
```

> 💡 **Pro tip**
> If you need a serializable value type in 2026, make it a **record**: compact, immutable, validated on read.

Q738 EX-059 • Intermediate

Why is serializing inner, anonymous and local classes a bad idea?

> ⚡ **Quick answer**
> Inner, anonymous and local classes that use the outer instance (or captured variables) carry the **outer instance (** `this$0` **)** and **captured values (** `val$x` **)** as synthetic fields, and they are serialized too. If the enclosing class isn't `Serializable`, you get `NotSerializableException: Outer`, and if it is, you serialize the **whole outer object graph** unintentionally. Their **synthetic names (** `Outer$1` **)** also change when source is reordered, breaking compatibility. Use **static nested** classes or top-level classes.

The second-to-last line shows the hidden cost: even when it works, an inner object serializes its **entire enclosing object**, with every field of it, which can be large or contain secrets. Static nested classes serialize only their own fields.

Related advice: don't make **lambdas and anonymous classes** your serialized callbacks (RMI, distributed executors, Spark closures, Hazelcast tasks): capture only what you need, use explicit named classes, and keep their package and name stable.

Java

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
out.writeObject(o); } return b.toByteArray(); }
static Object deser(byte[] bytes) throws Exception { try (ObjectInputStream in = new
ObjectInputStream(new ByteArrayInputStream(bytes))) { return in.readObject(); } }
static class Holder {                                                    // NOT serializable
    String label = "holder";
    class Inner implements Serializable { int v = 1; }
    Object anon() { return new Serializable() { int v = 2; String l = label; }; }          // uses
the outer: captures Holder.this
}
static class SafeHolder implements Serializable { String label = "safe"; class Inner implements
Serializable { int v = 3; } }
Holder h = new Holder();
```


© TechTribers. For personal use only. Do not redistribute.

```
for (Object o : new Object[]{h.new Inner(), h.anon()}) {
    try { ser(o); }
    catch (NotSerializableException e) { System.out.println(o.getClass().getName().replace("Main$",
"") + " -> NotSerializableException: " + e.getMessage().replace("Main$", "")); }
}
SafeHolder sh = new SafeHolder();
SafeHolder.Inner in = sh.new Inner();
System.out.println(((SafeHolder.Inner) deser(ser(in))).v + " " + ser(in).length + " bytes (the outer
is dragged along)");
static class Nested implements Serializable { int v = 4; }
System.out.println(ser(new Nested()).length + " bytes for a static nested class of the same shape");
```

▶ Output

```
Holder$Inner -> NotSerializableException: Holder
Holder$1 -> NotSerializableException: Holder
3 147 bytes (the outer is dragged along)
40 bytes for a static nested class of the same shape
```

⚠️ **Interview trap**

Frameworks with distributed tasks (Spark, Flink, Hazelcast) ship closures by serialization: capturing `this` there accidentally ships (or fails to ship) the whole enclosing object. Hoist what you need into local variables first.

🩺 **Q739** EX-060 • Intermediate

How do you debug NotSerializableException, and which JDK types are not serializable?

⚡ **Quick answer**

The message names the **class** that isn't `Serializable`, but not **where** it was found. Run with `-Dsun.io.serialization.extendedDebugInfo=true`: the stack trace then includes the **object path** (`field "owner" (class "Order")`, `object (class "Thread")` ...) to the offending object. Common culprits: loggers, threads and executors, streams, `Optional`, sockets and connections, lambdas without a serializable target, anonymous classes capturing an outer instance, and your own classes missing `implements Serializable` inside a collection field.

With the debug flag on (as in this run) the exception **message** carries the path from the root object to the failing one (`root object`, `field (class "Main$Order", name: "meta", type: "interface java.util.Map")`, `- object (class "java.util.HashMap")`, ...). **Without** the flag you only see `Plain`, and in a large graph (a session with hundreds of attributes) finding it is slow. The flag has a performance cost, so enable it for diagnosis only.

Fixes: make the class `Serializable` (or a proper proxy), mark the field `transient` and restore it in `readObject`/lazily, replace the type (store the **id** instead of the entity or connection), or stop serializing it at all (JSON/DTO).

```
static byte[] ser(Object o) throws Exception { ByteArrayOutputStream b = new
ByteArrayOutputStream(); try (ObjectOutputStream out = new ObjectOutputStream(b)) {
```


© TechTribers. For personal use only. Do not redistribute.

```
out.writeObject(o); } return b.toByteArray(); }
static class Plain { }
static class Order implements Serializable { String id = "o1"; List<Object> items = new ArrayList<>
(); Map<String, Object> meta = new HashMap<>(); }
Order o = new Order();
o.items.add("book");
o.meta.put("audit", new Plain());
try { ser(o); }
catch (NotSerializableException e) {
    String[] lines = e.getMessage().replace("Main$", "").replaceAll("@[0-9a-f]+",
"@hash").split("\n");       // with extendedDebugInfo the PATH is appended to the message
    System.out.println("class: " + lines[0]);
    for (int i = 1; i < lines.length; i++) System.out.println(lines[i].trim());
}
Map<String, Object> samples = new LinkedHashMap<>();
samples.put("Optional", Optional.of(1)); samples.put("Thread", new Thread()); samples.put("Object",
new Object()); samples.put("Logger", java.util.logging.Logger.getLogger("x"));
ExecutorService pool = Executors.newSingleThreadExecutor();
samples.put("ExecutorService", pool); samples.put("Scanner", new Scanner("x"));
samples.put("Random", new Random()); samples.put("AtomicInteger", new AtomicInteger());
samples.put("Date", new java.util.Date()); samples.put("ReentrantLock", new
java.util.concurrent.locks.ReentrantLock()); samples.put("List.of(1)", List.of(1));
samples.put("String", "text");
for (var e : samples.entrySet()) {
    try { ser(e.getValue()); System.out.println(e.getKey() + " serializable"); }
    catch (NotSerializableException ex) { System.out.println(e.getKey() + " NOT serializable"); }
}
pool.shutdown();
```

▶ Output

```
class: Plain
- custom writeObject data (class "java.util.HashMap")
- object (class "java.util.HashMap", {audit=Plain@hash})
- field (class "Order", name: "meta", type: "interface java.util.Map")
- root object (class "Order", Order@hash)
Optional NOT serializable
Thread NOT serializable
Object NOT serializable
Logger NOT serializable
ExecutorService NOT serializable
Scanner NOT serializable
Random serializable
AtomicInteger serializable
Date serializable
ReentrantLock serializable
List.of(1) serializable
String serializable
```

💡 **Pro tip**

Don't keep `Logger`, `Thread`, `ExecutorService`, connections or lambdas in serializable classes: make loggers `static final` (static fields aren't serialized) and everything else `transient`.

© TechTribers. For personal use only. Do not redistribute.

Series progress

739 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 16 starts at Q740.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50
Part 13	Q530–Q579	50
Part 14	Q580–Q679	100
Part 15	Q680–Q739	60

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

TechTribers TechTribers

© TechTribers. For personal use only. Do not redistribute.